

Học sâu và ứng dụng

Tiếp cận mạng nơ-ron

Giảng viên: TS. Phạm Đình Phong

Email: phongpd@utc.edu.vn

Phone: 0972481813

Tiếp cận mạng nơ-ron (neural network)

- Con chó có thể phân biệt được người thân trong gia đình và người lạ hay đứa trẻ có thể phân biệt được các con vật.
- Những việc tưởng chừng như rất đơn giản như vậy nhưng lại cực kì khó để thực hiện bằng máy tính.
- Vậy sự khác biệt nằm ở đâu? Câu trả lời nằm ở cấu trúc bộ não với lượng lớn các nơ-ron thần kinh được liên kết với nhau.
- Liệu máy tính có thể mô phỏng lại cấu trúc bộ não để giải các bài toán trên?

Tiếp cận mạng nơ-ron (neural network)

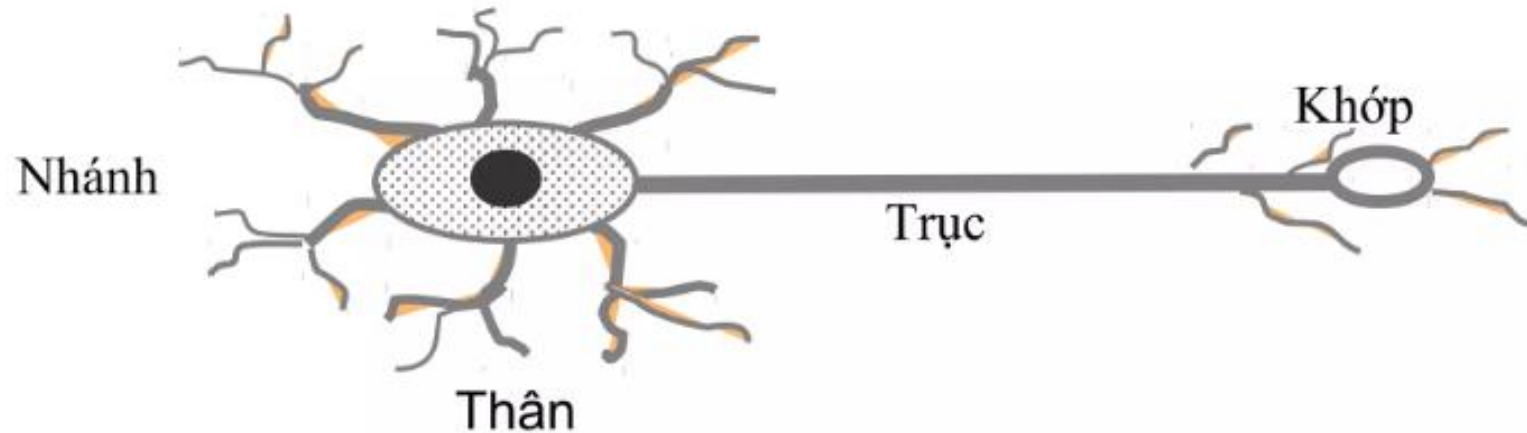
- Hoạt động của các nơ-ron
 - Nơ-ron là đơn vị cơ bản cấu tạo hệ thống thần kinh và là thành phần quan trọng nhất của não. Đầu chúng ta gồm khoảng 10 triệu nơ-ron và mỗi nơ-ron lại liên kết với tầm 10.000 nơ-ron khác.
 - Ở mỗi nơ-ron có phần thân (soma) chứa nhân, các tín hiệu đầu vào qua sợi nhánh (dendrites) và các tín hiệu đầu ra qua sợi trục (axon) kết nối với các nơ-ron khác. Hiểu đơn giản mỗi nơ-ron nhận dữ liệu đầu vào qua sợi nhánh và truyền dữ liệu đầu ra qua sợi trục, đến các sợi nhánh của các nơ-ron khác.

Tiếp cận mạng nơ-ron (neural network)

- Hoạt động của các nơ-ron
 - Mỗi nơ-ron nhận xung điện từ các nơ-ron khác qua sợi nhánh. Nếu các xung điện này đủ lớn để kích hoạt nơ-ron thì tín hiệu này đi qua sợi trục đến các sợi nhánh của các nơ-ron khác. => Ở mỗi nơ-ron cần quyết định có kích hoạt nơ-ron đấy hay không. Tương tự các hoạt động của hàm sigmoid bài trước.
 - Tuy nhiên, tiếp cận mạng nơ-ron chỉ là lấy cảm hứng từ não bộ và cách nó hoạt động, chứ không phải bắt chước toàn bộ các chức năng của nó. Việc chính của chúng ta là dùng mô hình đó đi giải quyết các bài toán cần thiết.

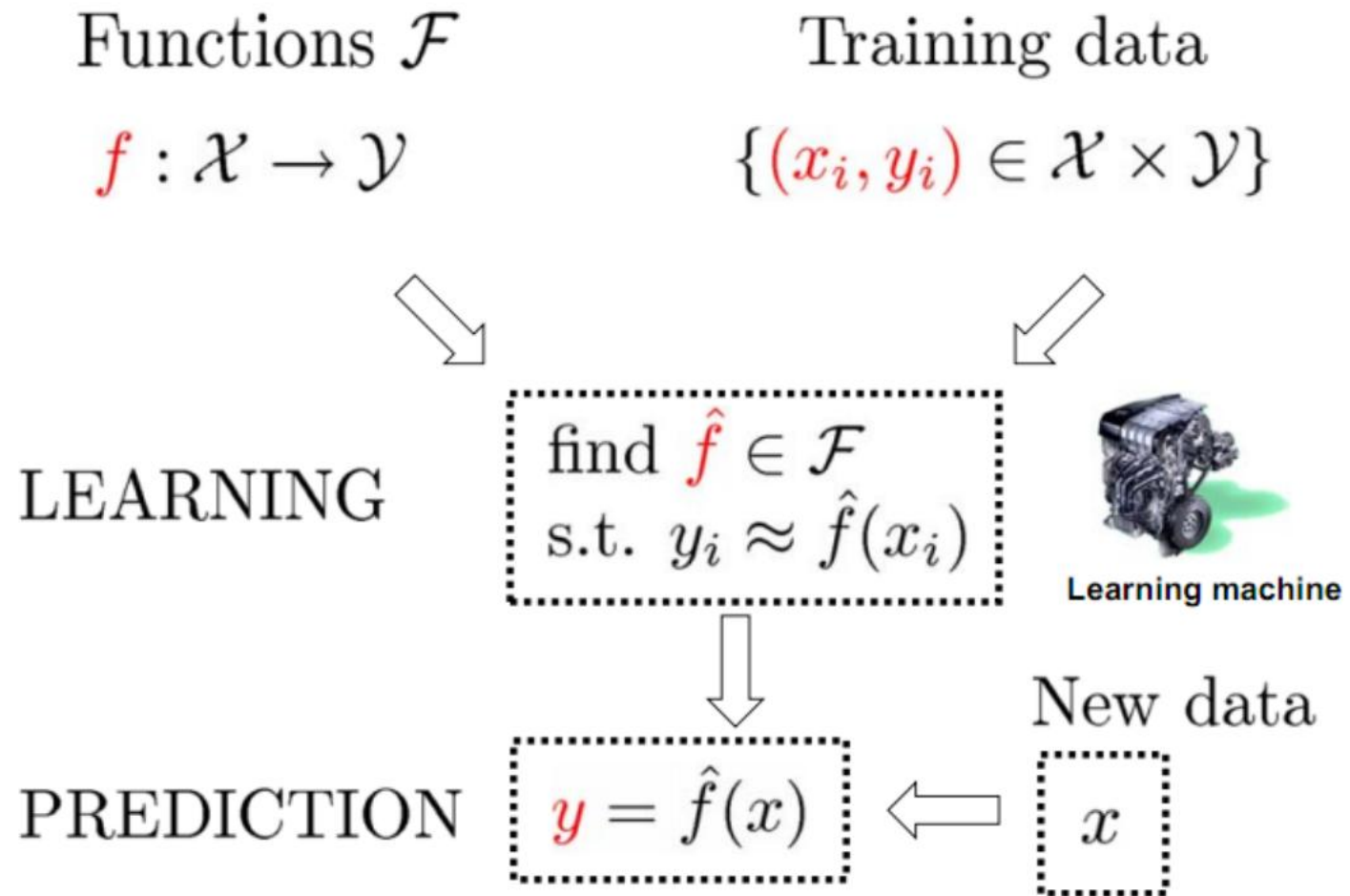
Tiếp cận mạng nơ-ron

- **Mạng nơ-ron (neural network):** Là phương pháp giải quyết bài toán trên máy tính mô phỏng theo hoạt động của các tế bào thần kinh trong não bộ.
- **Mạng nơ-ron nhân tạo:** là sự mô phỏng cấu trúc của mạng nơ-ron sinh học. Mạng nơ-ron nhân tạo được tạo bởi sự kết nối của rất nhiều đơn vị thần kinh được gọi là perceptron



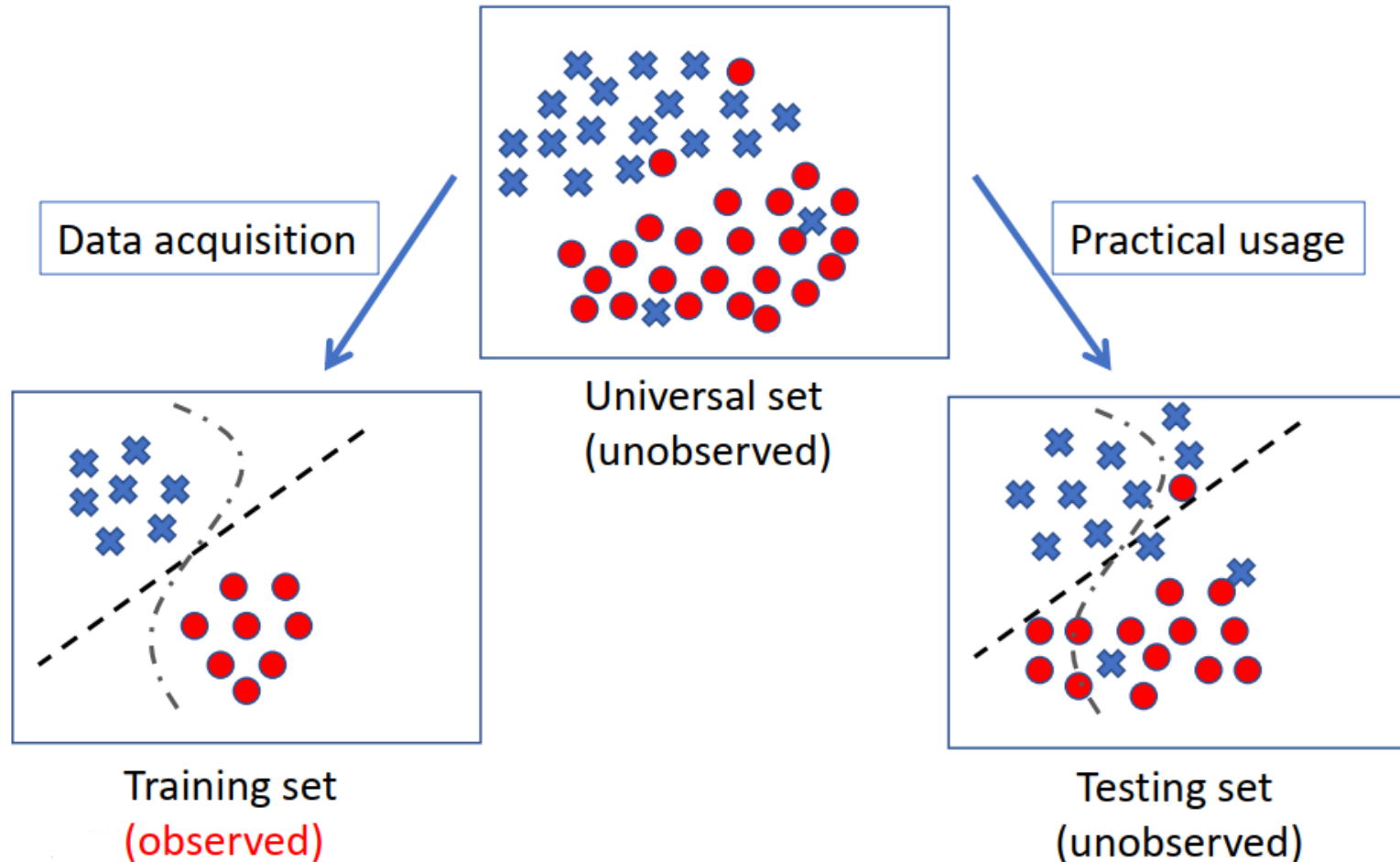
Tiếp cận mạng nơ-ron (neural network)

- Học máy có giám sát



Tiếp cận mạng nơ-ron (neural network)

- Tập huấn luyện và tập kiểm tra

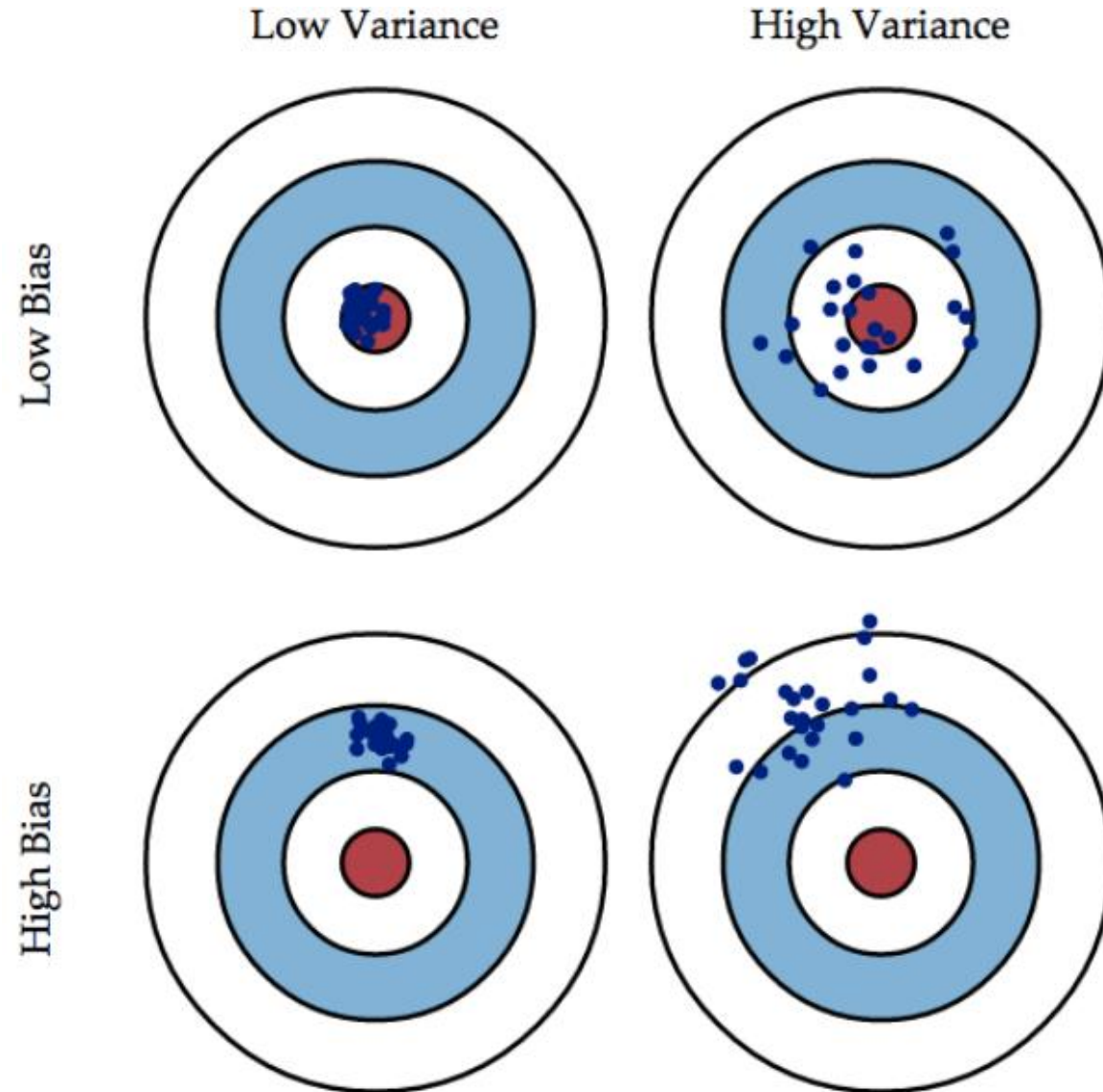


Tiếp cận mạng nơ-ron (neural network)

- Hiện tượng overfit và underfit
 - **Underfitting**: mô hình quá “đơn giản” để biểu diễn các tính chất của dữ liệu
 - Bias cao và variance thấp
 - Sai số cao trên tập huấn luyện và tập kiểm tra
 - **Overfitting**: mô hình quá “phức tạp” dẫn tới học cả nhiễu trong dữ liệu
 - Bias thấp và variance cao
 - Sai số thấp trên tập huấn luyện và sai số cao trên tập kiểm tra

Tiếp cận mạng nơ-ron (neural network)

- Minh họa Bias-Variance



Mô hình mạng nơ-ron (neural network)

- Logistic regression
 - Xét Logistic regression với mô hình neural network đơn giản nhất chỉ với input layer và output layer.
 - Mô hình của logistic regression từ bài trước là:

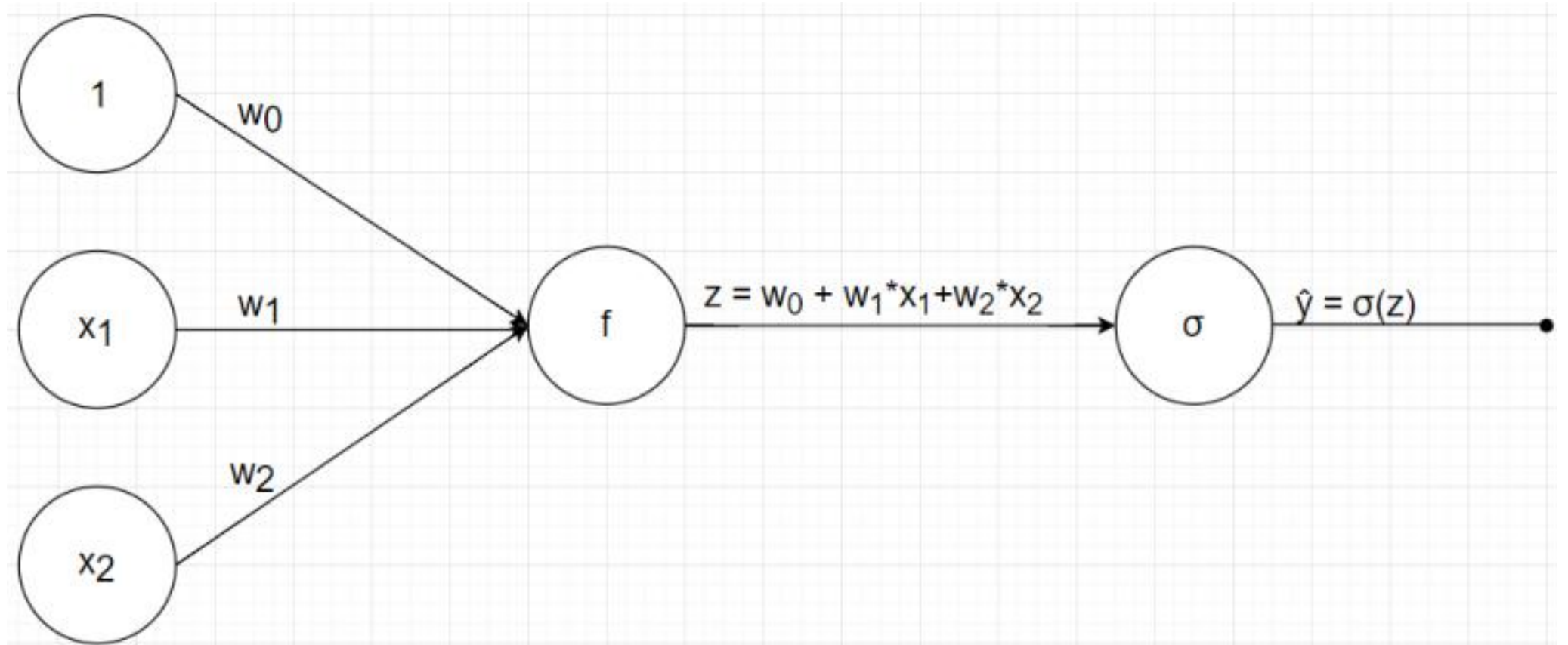
$$\hat{y}_i = \sigma(w_0 + w_1 * x_1 + w_2 * x_2)$$

Có 2 bước:

- Tính tổng linear: $z = 1 * w_0 + w_1 * x_1 + w_2 * x_2$
- Áp dụng sigmoid function: $\hat{y}_i = \sigma(z)$

Mô hình mạng nơ-ron (neural network)

- Logistic regression



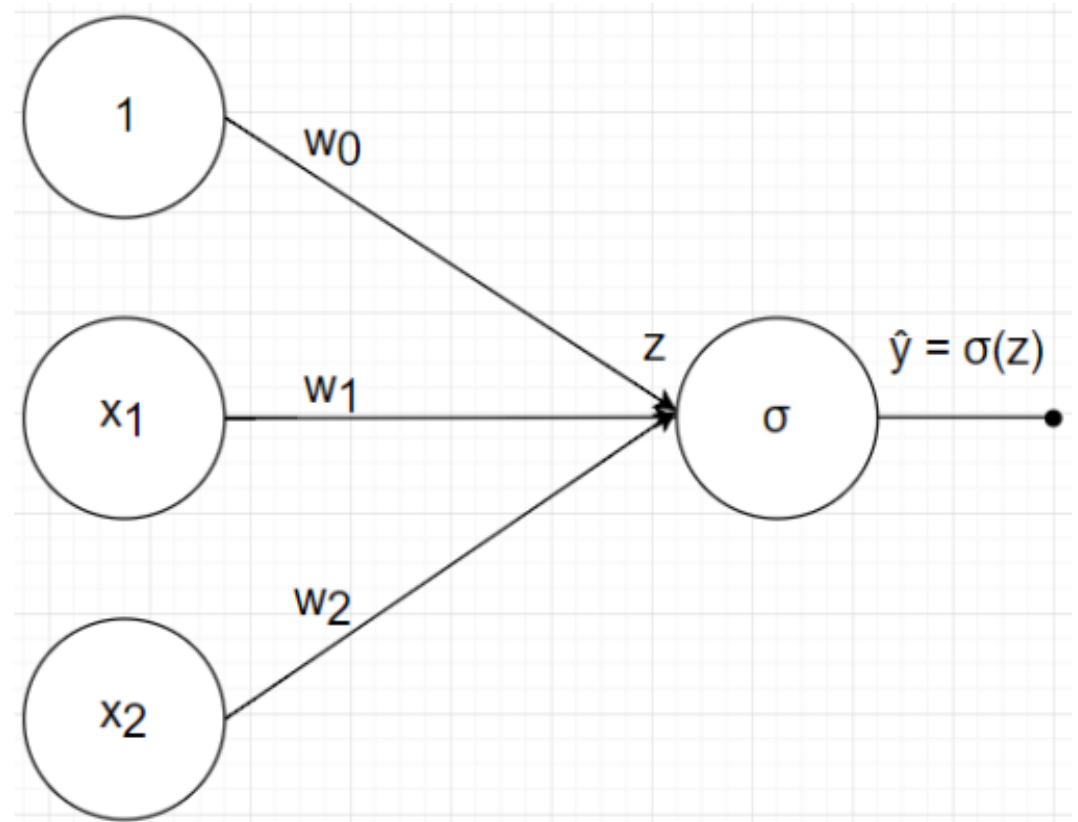
Mô hình mạng nơ-ron (neural network)

- Logistic regression

- Để biểu diễn gọn lại ta sẽ gộp hai bước trên thành một trên biểu đồ như hình sau:

- Hệ số w_0 được gọi là bias.
- Hàm sigmoid ở đây được gọi là hàm kích hoạt (activation function).

Lưu ý: nếu bỏ w_0 thì phương trình đường phương trình giờ có dạng:
 $y = w_1 * x$ sẽ luôn đi qua gốc tọa độ và không tổng quát hóa được phương trình đường thẳng



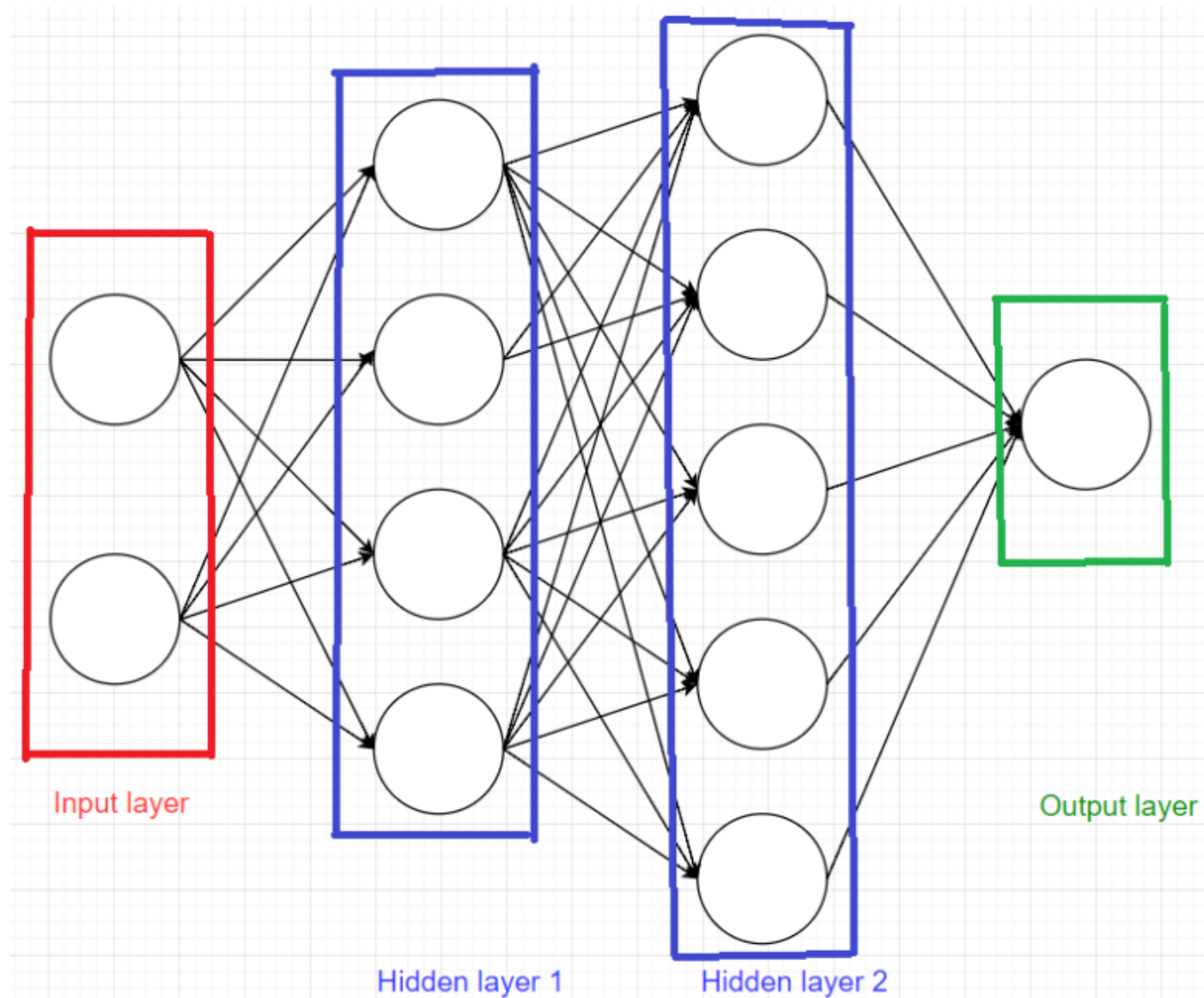
Mô hình mạng nơ-ron (neural network)

- Mô hình tổng quát
 - Lớp (Layer) đầu tiên là input layer, các layer ở giữa được gọi là hidden layer, layer cuối cùng được gọi là output layer. Các hình tròn được gọi là nút (node).
 - Mỗi mô hình luôn có 1 input layer, 1 output layer, có thể có hoặc không các hidden layer. Tổng số layer trong mô hình được quy ước là số layer - 1 (không tính input layer).
 - Mỗi node trong hidden layer và output layer:
 - Liên kết với tất cả các node ở layer trước đó với các hệ số w riêng.
 - Mỗi node có 1 hệ số bias b riêng.
 - Diễn ra 2 bước: tính tổng linear và áp dụng activation function.

Mô hình mạng nơ-ron (neural network)

- Mô hình tổng quát

Ví dụ, như ở hình bên có 1 input layer, 2 hidden layer và 1 output layer. Số lượng layer của mô hình là 3 layer.



Mô hình mạng nơ-ron (neural network)

- Kí hiệu

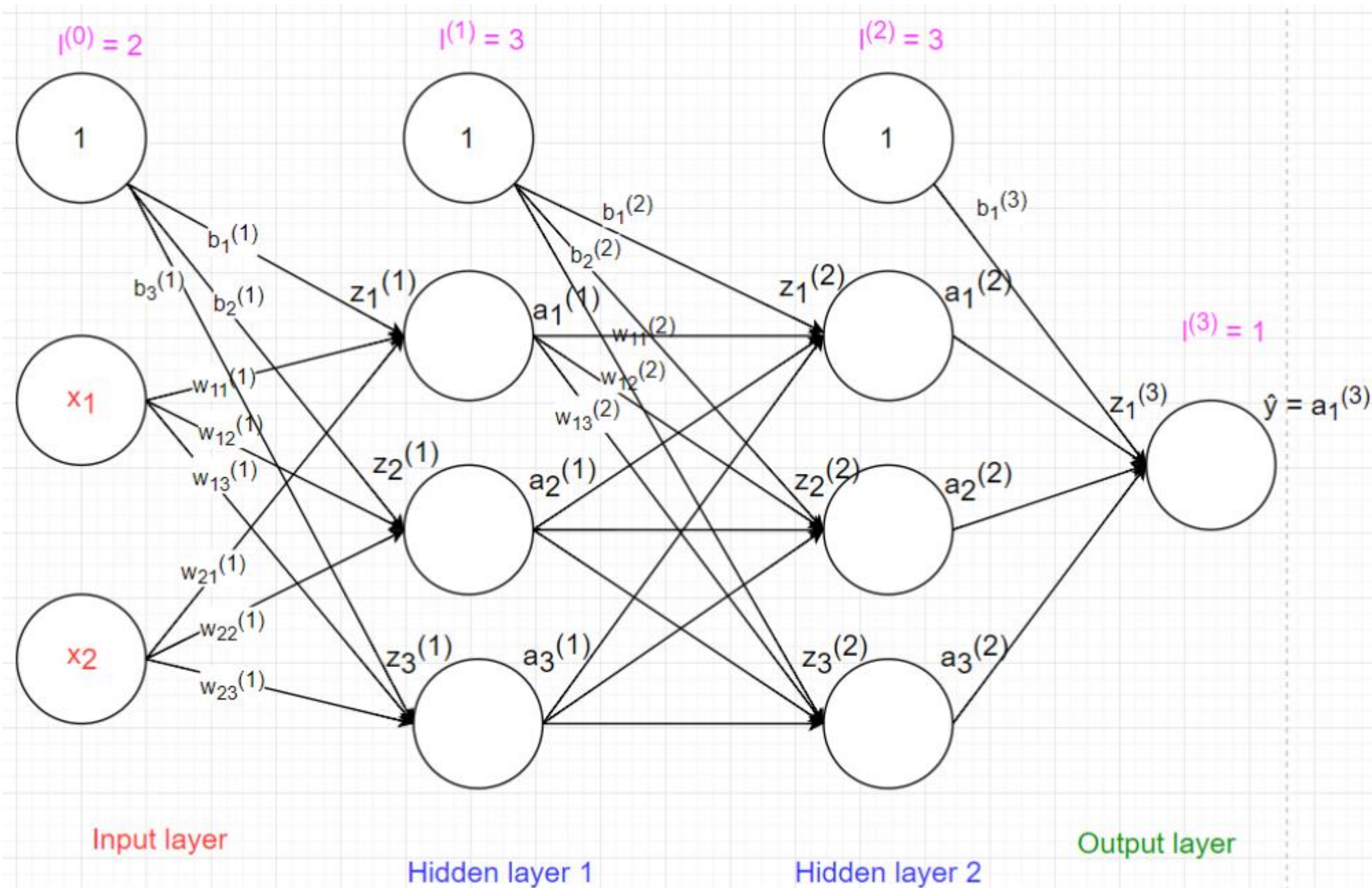
- Số node trong hidden layer thứ i là $l^{(i)}$.
- Ma trận W_k kích thước $l^{(k-1)} * l^{(k)}$ là ma trận hệ số giữa layer $k - 1$ và layer k , trong đó $w_{ij}^{(k)}$ là hệ số kết nối từ node thứ i của layer $k - 1$ đến node thứ j của layer k .
- Vector $b^{(k)}$ kích thước $l^{(k)} * 1$ là hệ số bias của các node trong layer k , trong đó $b_i^{(k)}$ là bias của node thứ i trong layer k .

Mô hình mạng nơ-ron (neural network)

- Kí hiệu

- Số node trong hidden layer thứ i là $l^{(i)}$.
- Với node thứ i trong layer l có bias $b_i^{(l)}$ thực hiện 2 bước:
 - Tính tổng linear: $z_i^{(l)} = \sum_{j=1}^{l^{(l-1)}} a_j^{(l-1)} * w_{ji}^{(l)} + b_i^{(l-1)}$ là tổng tất cả các node trong layer trước nhân với hệ số w tương ứng, rồi cộng với bias b .
 - Áp dụng activation function: $a_i^{(l)} = \sigma(z_i^{(l)})$
- Vector $z^{(k)}$ kích thước $l^{(k)} * 1$ là giá trị các node trong layer k sau bước tính tổng linear.
- Vector $a^{(k)}$ kích thước $l^{(k)} * 1$ là giá trị của các node trong layer k sau khi áp dụng hàm activation function.

- Kí hiệu



Mô hình mạng nơ-ron (neural network)

- Mô hình mạng nơ-ron trên gồm 3 layer. Input layer có 2 node ($l^{(0)} = 2$), **hidden layer 1 có 3 node**, **hidden layer 2 có 3 node** và **output layer có 1 node**.
- Do mỗi node trong hidden layer và output layer đều có bias nên trong input layer và hidden layer cần thêm node 1 để tính bias (nhưng không tính vào tổng số node mà layer có).

Tại node thứ 2 ở layer 1, ta có:

- $z_2^{(1)} = x_1 * w_{12}^{(1)} + x_2 * w_{22}^{(1)} + b_2^{(1)}$
- $a_2^{(1)} = \sigma(z_2^{(1)})$

Hay ở node thứ 3 layer 2, ta có:

- $z_3^{(2)} = a_1^{(1)} * w_{13}^{(2)} + a_2^{(1)} * w_{23}^{(2)} + a_3^{(1)} * w_{33}^{(2)} + b_3^{(2)}$
- $a_3^{(2)} = \sigma(z_3^{(2)})$

Mô hình mạng nơ-ron (neural network)

- Feed forward
 - Để nhất quán về mặt ký hiệu, gọi input layer là $a^{(0)} (= x)$ kích thước 2×1 .

$$\begin{aligned} z^{(1)} &= \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \\ z_3^{(1)} \end{bmatrix} = \begin{bmatrix} a_1^{(0)} * w_{11}^{(1)} + a_2^{(0)} * w_{21}^{(1)} + a_3^{(0)} * w_{31}^{(1)} + b_1^{(1)} \\ a_1^{(0)} * w_{12}^{(1)} + a_2^{(0)} * w_{22}^{(1)} + a_3^{(0)} * w_{32}^{(1)} + b_2^{(1)} \\ a_1^{(0)} * w_{13}^{(1)} + a_2^{(0)} * w_{23}^{(1)} + a_3^{(0)} * w_{33}^{(1)} + b_3^{(1)} \end{bmatrix} \\ &= (W^{(1)})^T * a^{(0)} + b^{(1)} \end{aligned}$$

$$a^{(1)} = \sigma(z^{(1)})$$

Mô hình mạng nơ-ron (neural network)

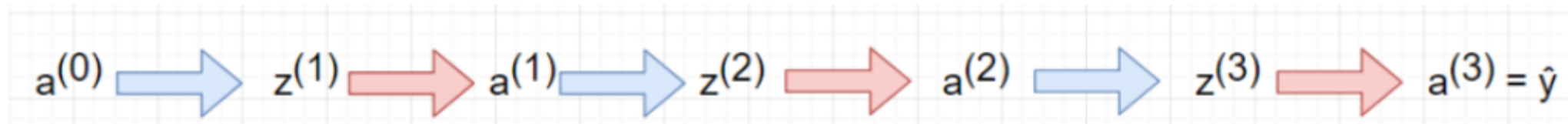
- Feed forward
 - Tương tự ta có:

$$z^{(2)} = (W^{(2)})^T * a^{(1)} + b^{(2)}$$

$$a^{(2)} = \sigma(z^{(2)})$$

$$z^{(3)} = (W^{(3)})^T * a^{(2)} + b^{(3)}$$

$$\hat{y} = a^{(3)} = \sigma(z^{(3)})$$



Mô hình mạng nơ-ron (neural network)

- Biểu diễn dưới dạng ma trận

- Khi làm việc với dữ liệu ta cần tính dự đoán cho nhiều dữ liệu một lúc, nên gọi X là ma trận $n \times d$, trong đó n là số dữ liệu và d là số trường trong mỗi dữ liệu, trong đó $x_j^{[i]}$ là giá trị trường dữ liệu thứ j của dữ liệu thứ i . Ví dụ dataset bài trước thì: $n = 4, d = 2, x_1^{[1]} = 10, x_2^{[1]} = 1, x_1^{[3]} = 7, x_2^{[2]} = 2$

Lương	Thời gian làm việc
10	1
5	2
7	0.15
6	1.8

$$X = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_d^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_d^{[2]} \\ \dots & \dots & \dots & \dots \\ x_1^{[n]} & x_2^{[n]} & \dots & x_d^{[n]} \end{bmatrix} = \begin{bmatrix} -(x^{[1]})^T - \\ -(x^{[2]})^T - \\ \dots \\ -(x^{[n]})^T - \end{bmatrix}$$

Biểu diễn dạng ma trận của dữ liệu nhiều chiều trong dataset

Mô hình mạng nơ-ron (neural network)

- Biểu diễn dưới dạng ma trận
 - Do $x^{[1]}$ là vector kích thước $d \times 1$, tuy nhiên, ở X mỗi dữ liệu được viết theo hàng nên cần transpose $x^{[1]}$ thành kích thước $1 \times d$, kí hiệu: $-(x^{[1]})^T -$
 - Gọi ma trận $Z^{(i)}$ kích thước $N \times l^{(i)}$, trong đó $z_j^{(i)[k]}$ là giá trị thứ j trong layer i sau bước tính tổng linear của dữ liệu thứ k trong dataset.
 - Tương tự, gọi ma trận $A^{(i)}$ kích thước $N \times l^{(i)}$ trong đó $a_j^{(i)[k]}$ là giá trị thứ j trong layer i sau khi áp dụng activation function của dữ liệu thứ k trong dataset.

*** Kí hiệu (i) là layer thứ i và kí hiệu $[k]$ là dữ liệu thứ k trong dataset.

Mô hình mạng nơ-ron (neural network)

- Biểu diễn dưới dạng ma trận

$$Z^{(i)} = \begin{bmatrix} z_1^{(i)[1]} & z_2^{(i)[1]} & \dots & z_{l^{(i)}}^{(i)[1]} \\ z_1^{(i)[2]} & z_2^{(i)[2]} & \dots & z_{l^{(i)}}^{(i)[2]} \\ \dots & \dots & \dots & \dots \\ z_1^{(i)[n]} & z_2^{(i)[n]} & \dots & z_{l^{(i)}}^{(i)[n]} \end{bmatrix} = \begin{bmatrix} - (z^{(i)[1]})^T - \\ - (z^{(i)[2]})^T - \\ \dots \\ - (z^{(i)[n]})^T - \end{bmatrix}$$

Mô hình mạng nơ-ron (neural network)

$$\begin{aligned} Z^{(1)} &= \begin{bmatrix} (z^{(1)[1]})^T \\ (z^{(1)[2]})^T \\ \dots \\ (z^{(1)[n]})^T \end{bmatrix} = \begin{bmatrix} (x^{[1]})^T * w^{(1)} + (b^{(1)})^T \\ (x^{[2]})^T * w^{(1)} + (b^{(1)})^T \\ \dots \\ (x^{[n]})^T * w^{(1)} + (b^{(1)})^T \end{bmatrix} = \\ &X * W^{(1)} + \begin{bmatrix} (b^{(1)})^T \\ (b^{(1)})^T \\ \dots \\ (b^{(1)})^T \end{bmatrix} \\ &= X * W^{(1)} + b^{(1)} \end{aligned}$$

Mô hình mạng nơ-ron (neural network)

$$A^{(1)} = \sigma(Z^{(1)})$$

$$Z^{(2)} = A^{(1)} * W^{(2)}, A^{(2)} = \sigma(Z^{(2)})$$

$$Z^{(3)} = A^{(2)} * W^{(3)}$$

$$\hat{Y} = A^{(3)} = \sigma(Z^{(3)})$$

- Ta có thể tính được giá trị dự đoán của nhiều dữ liệu một lúc dưới dạng ma trận. Tức là, từ input X ta có thể tính được giá trị dự đoán $\hat{Y}$.

Mô hình mạng nơ-ron (neural network)

- Công việc chính cần làm là đi tìm hệ số W và b .
- Có thể sử dụng thuật toán **gradient descent** và việc quan trọng nhất trong thuật toán gradient descent là đi tìm **đạo hàm** của các hệ số đối với **loss function**.
- Việc tính **đạo hàm** của các hệ số trong mạng nơ-ron được thực hiện bởi thuật toán lan truyền ngược (back propagation).

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - AND, OR, XOR là các phép toán thực hiện phép tính trên bit, trong đó mỗi bit nhận 1 trong 2 giá trị là 0 hoặc 1.
 - Phép tính NOT của 1 bit cho ra giá trị ngược lại.

A	NOT(A)
0	1
1	0

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - AND: Phép tính AND của 2 bit cho giá trị 1 nếu cả 2 bit bằng 1 và cho giá trị bằng 0 trong các trường hợp còn lại. Bảng chân lý

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

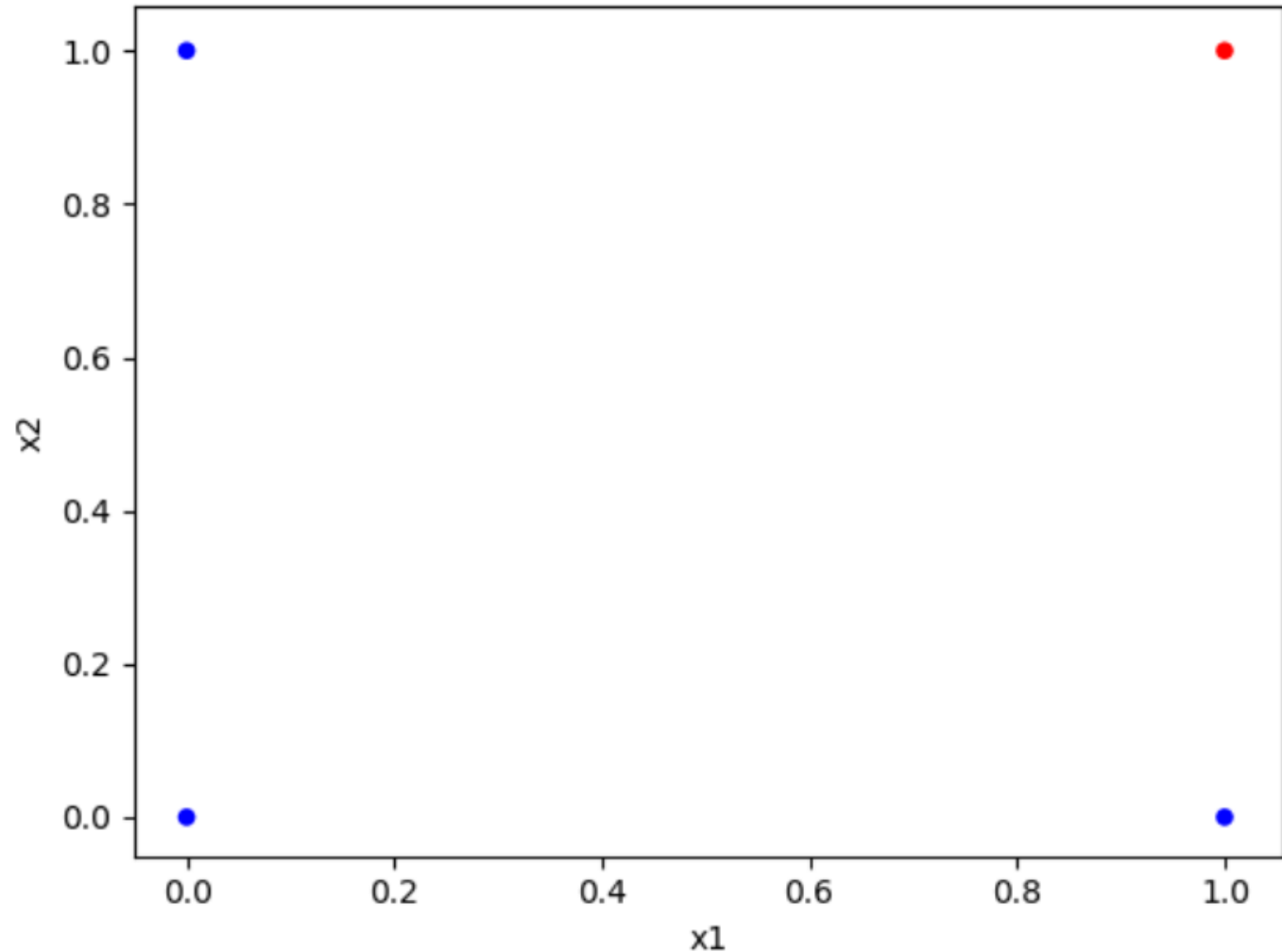
- Giờ muốn máy tính học toán tử AND, ta thấy là kết quả là 0 và 1, nên nghĩ ngay đến logistic regression với dữ liệu

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - AND:

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Chấm xanh là giá trị 0,
chấm đỏ là giá trị 1



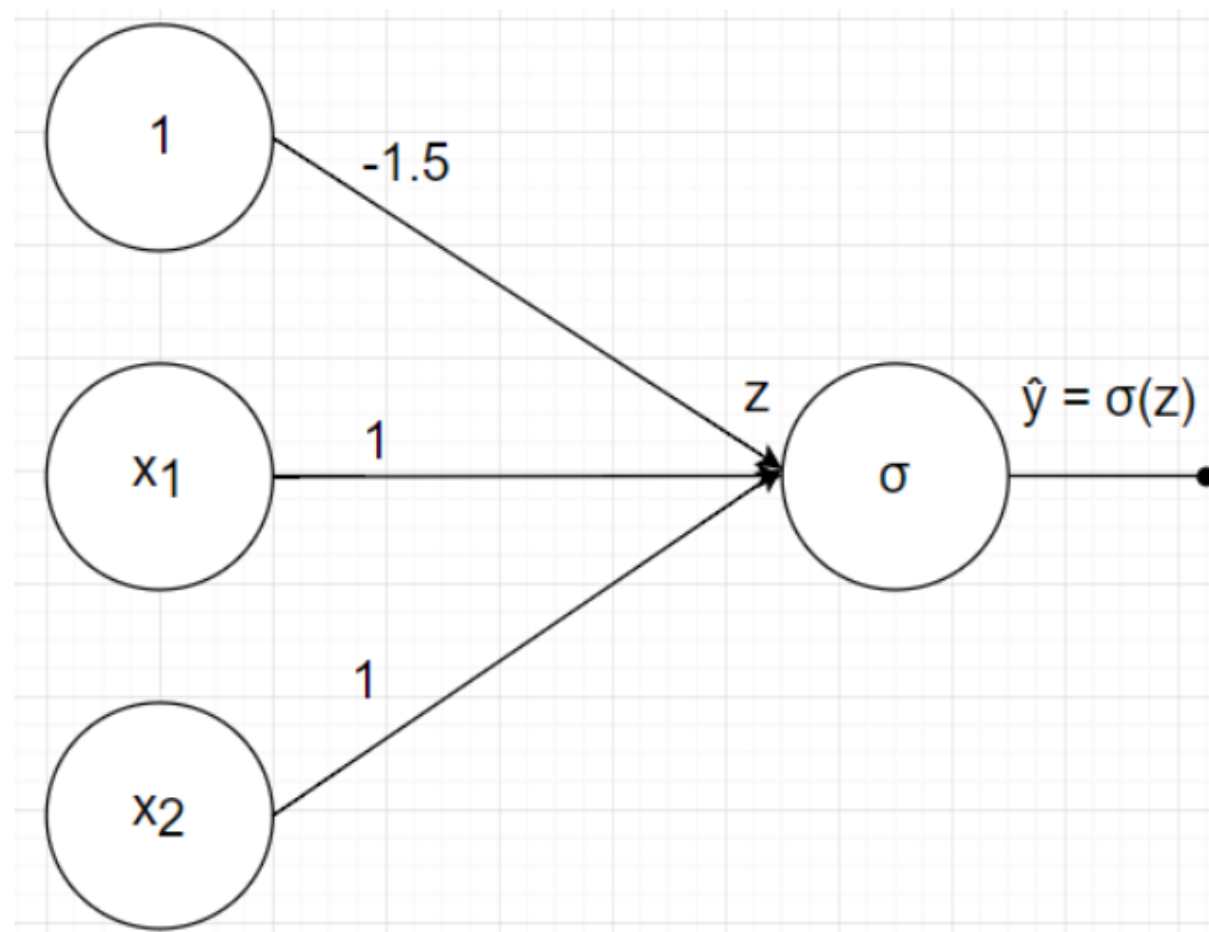
Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR

- AND:

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Logistic regression chính là đường thẳng phân chia giữa các điểm, khi áp dụng thuật toán trong bài logistic regression ta tìm được $w_0 = -1.5$, $w_1 = 1$, $w_2 = 1$

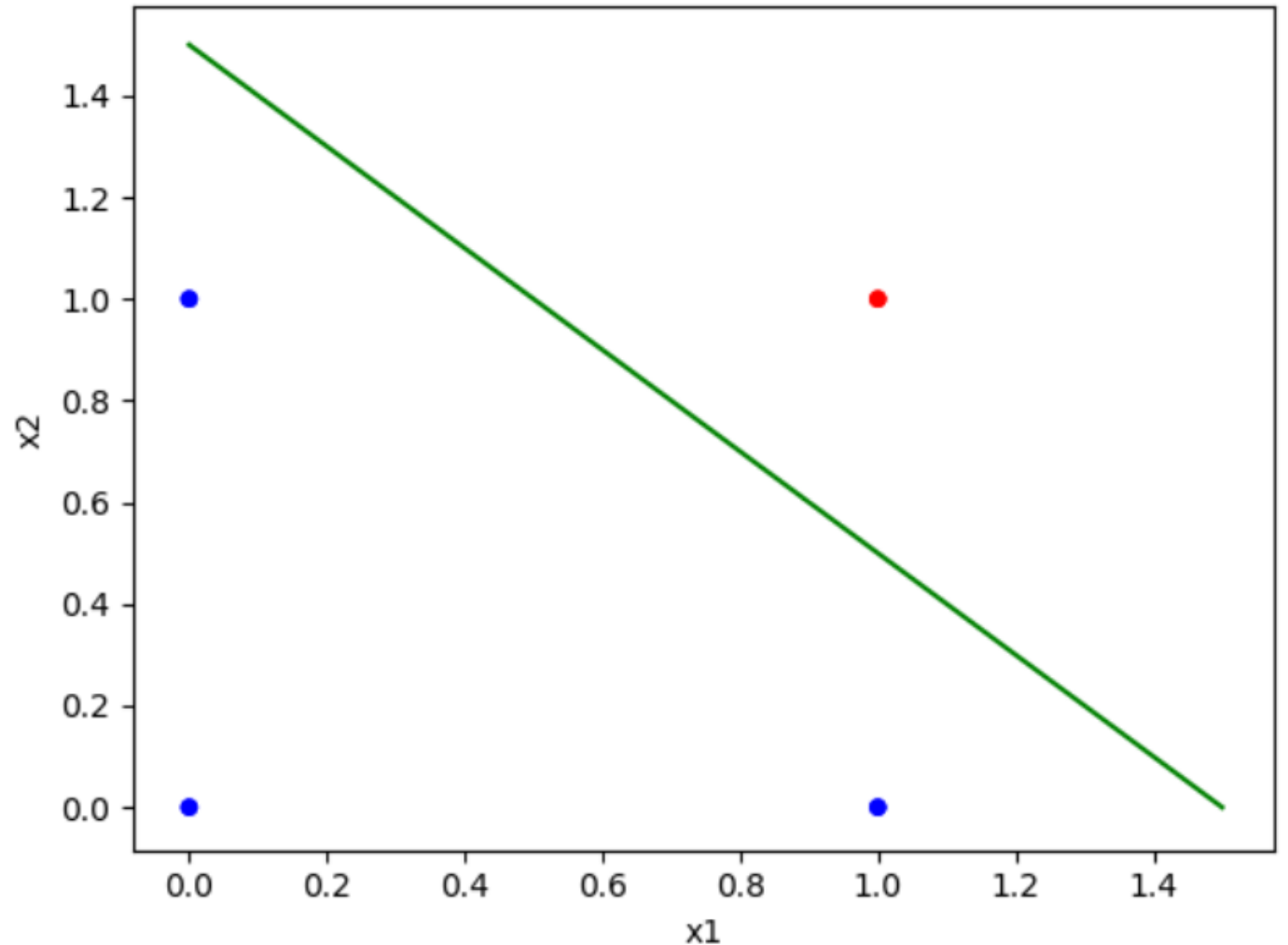


x_1 AND x_2

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - AND:

Đường thẳng $y = 1.5 - x$ phân chia hai miền dữ liệu



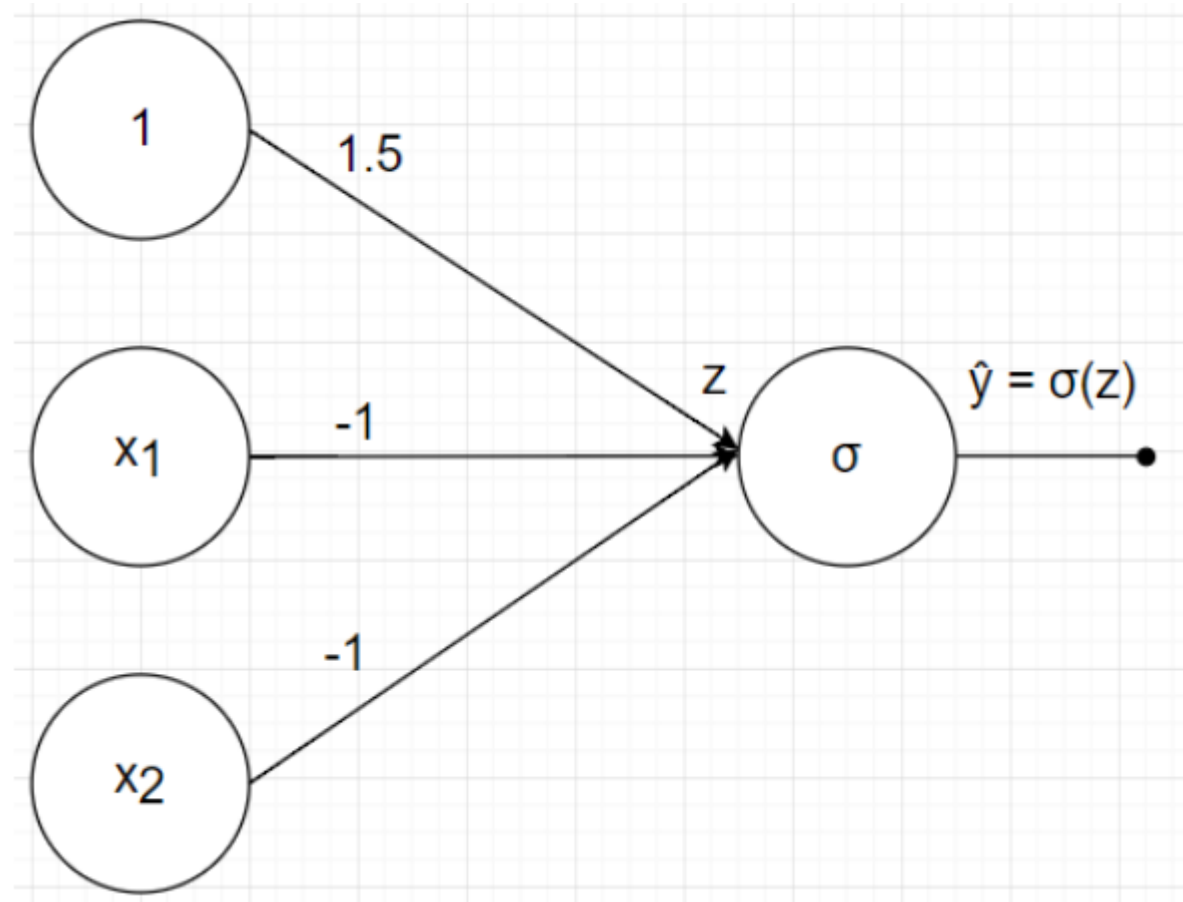
Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - AND:
 - Nhận xét, do phép tính NOT là đổi giá trị của bit, nên phép tính NOT(A AND B) có thể biểu diễn như hình trên với việc đổi màu các điểm từ đỏ thành xanh và xanh thành đỏ. Do đó đường phân chia không thay đổi và hai miền giá trị đổi dấu cho nhau => giá trị các tham số đổi dấu $w_0 = 1.5$, $w_1 = -1$, $w_2 = -1$

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - AND:

NOT (x_1 AND x_2)



Mô hình mạng nơ-ron (neural network)

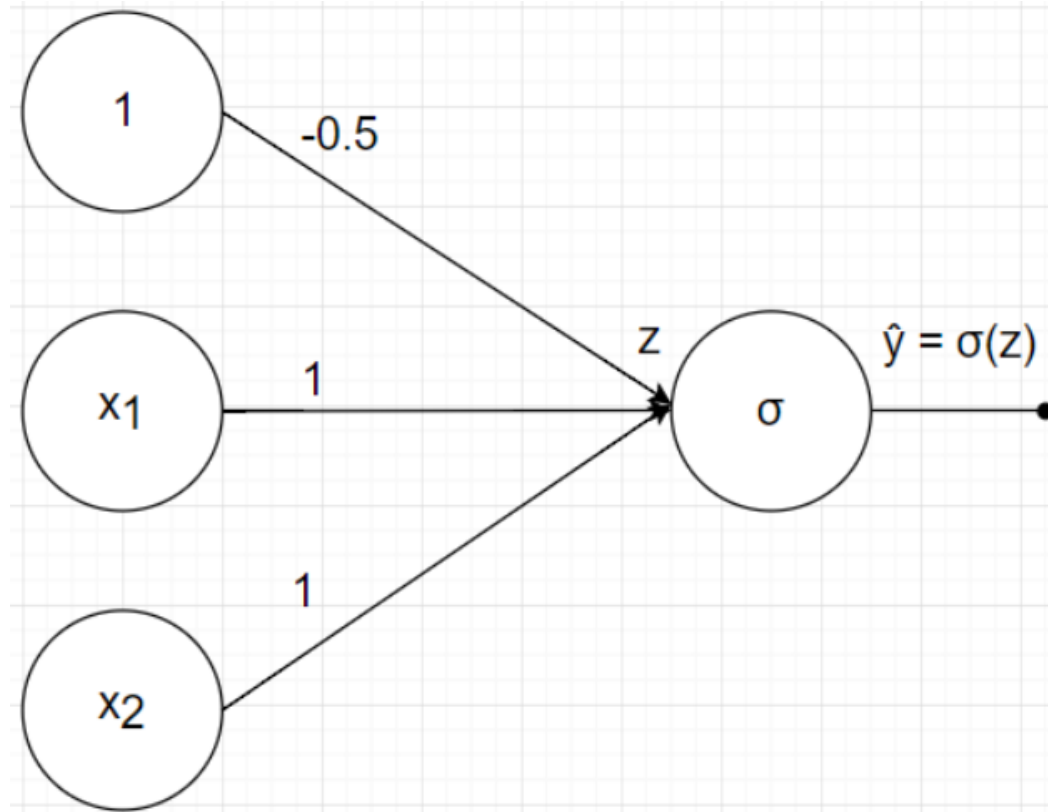
- Logistic regression với toán tử XOR
 - OR:
 - Phép tính OR của 2 bit cho giá trị 1 nếu 1 trong 2 bit bằng 1 và cho giá trị bằng 0 trong các trường hợp còn lại. Bảng giá trị chân lý:

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - OR:
 - Tương tự ta cũng tìm được $w_0 = -0.5$, $w_1 = 1$, $w_2 = 1$

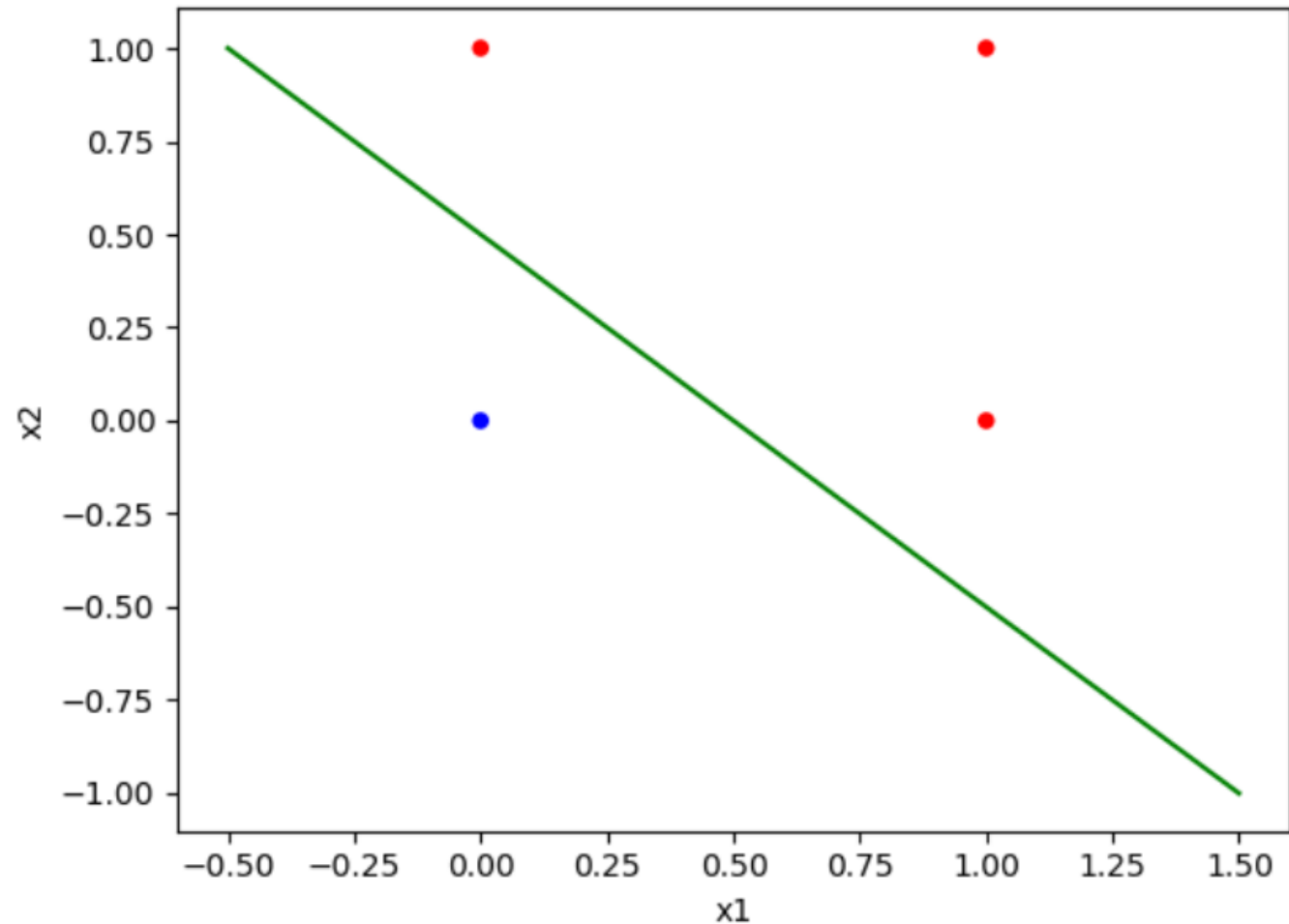
x_1 OR x_2



Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - OR:

x_1 OR x_2



Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR

- XOR:

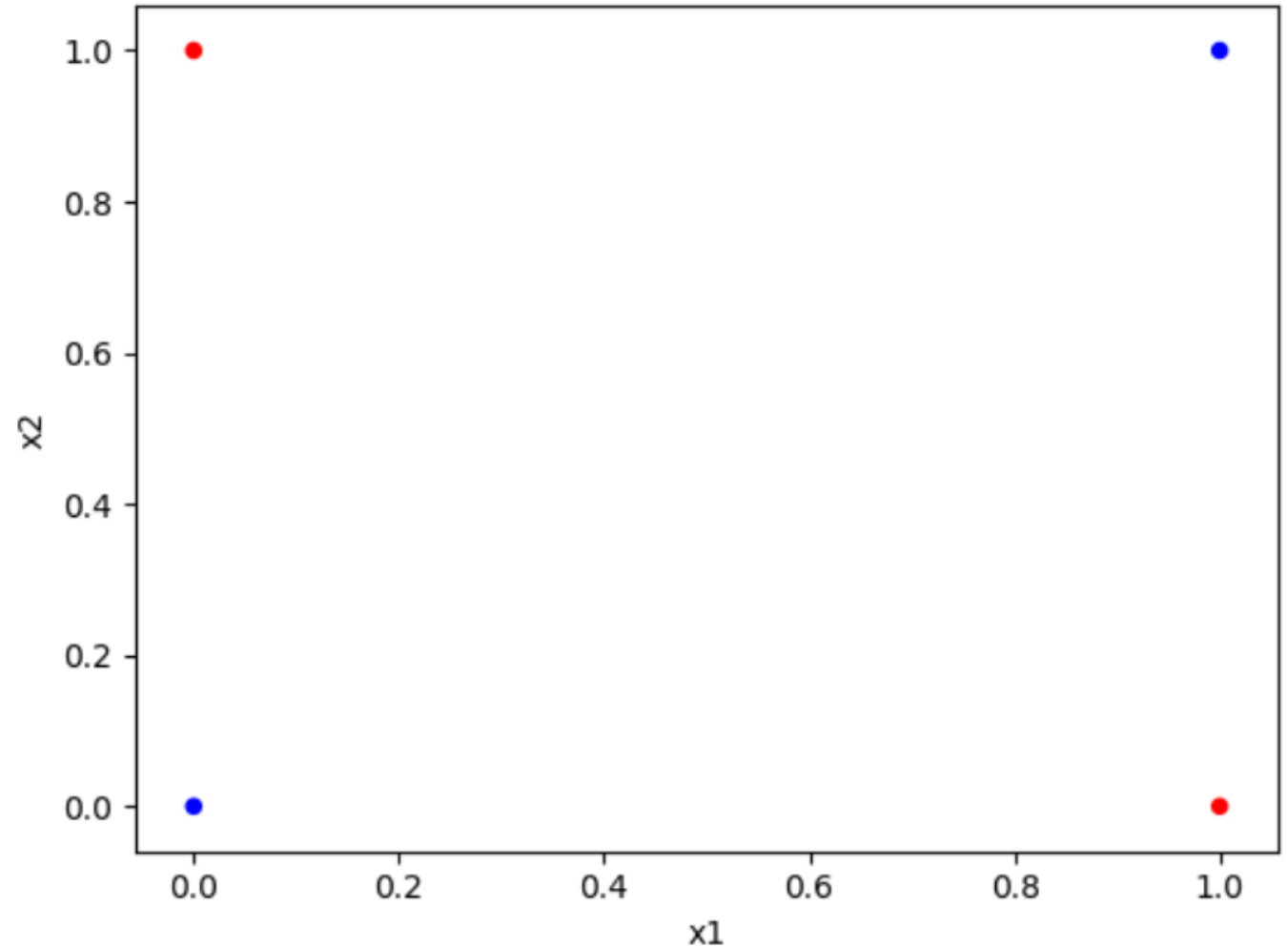
- Phép tính XOR của 2 bit cho giá trị 1 nếu đúng 1 trong 2 bit bằng 1 và cho giá trị bằng 0 trong các trường hợp còn lại. Bảng giá trị chân lý:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - XOR:

Khi thiết lập bài toán logistic regression, ta có đồ thị



Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - XOR:
 - Rõ ràng là không thể dùng một đường thẳng để phân chia dữ liệu thành 2 miền.
 - Khi dùng gradient descent vào bài toán XOR thì bất kể chạy bước 2 bao nhiêu lần hay chỉnh learning_rate thế nào thì vẫn không ra được kết quả như mong muốn.
 - Logistic regression như bài trước không thể giải quyết được vấn đề này, giờ cần một giải pháp mới !!!

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - XOR:
 - Áp dụng các kiến thức về bit ở trên, ta có:

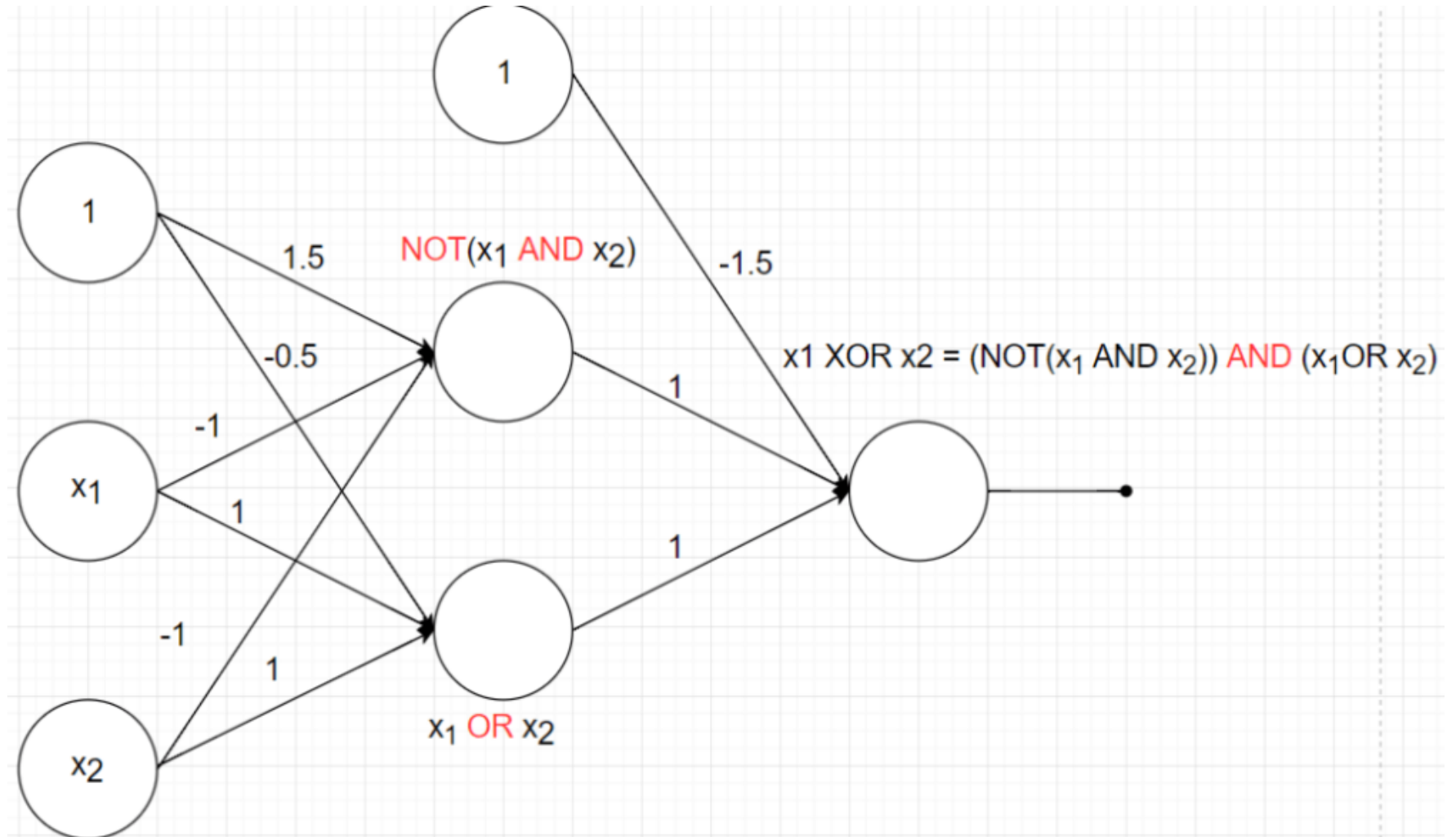
A	B	A XOR B	A AND B	NOT (A AND B)	A OR B	(NOT(A AND B) AND (A OR B))
0	0	0	0	1	0	0
0	1	1	0	1	1	1
1	0	1	0	1	1	1
1	1	0	1	0	1	0

- Do đó: $A \text{ XOR } B = (\text{NOT}(A \text{ AND } B) \text{ AND } (A \text{ OR } B))$, vậy để tính được XOR ta kết hợp NOT(AND) và OR sau đó tính phép tính AND.

Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - XOR:

Mô hình XOR



Mô hình mạng nơ-ron (neural network)

- Logistic regression với toán tử XOR
 - XOR:
 - Nhìn có vẻ rồi nhưng cùng phân tích:
 - node $\text{NOT}(x_1 \text{ AND } x_2)$ chính là từ hình trên, với 3 mũi tên chỉ đến từ 1, x_1 , x_2 với hệ số w_0 , w_1 , w_2 tương ứng là 1.5, -1, -1.
 - node tính $x_1 \text{ OR } x_2$ là từ hình trên.
 - node trong output layer là phép tính AND từ 2 node của layer trước, giá trị hệ số từ hình 1 mang xuống.
 - Nhận xét: mô hình logistic regression không giải quyết được bài toán XOR nhưng mô hình mới thì giải quyết được bài toán XOR. Đây là sự khác nhau:
 - Logistic regression chỉ có input layer và output layer
 - Mô hình mới có 1 hidden layer có 2 node ở giữa input layer và output layer.

=> Càng nhiều layer và node thì càng giải quyết được các bài toán phức tạp hơn.

Mô hình mạng nơ-ron (neural network)

- Lan truyền ngược (Backpropagation)
 - Bài trước đã học về mô hình neural network và feedforward
 - Nhiệm vụ là đi tìm hệ số W và b
 - Có thể sử dụng thuật toán gradient descent và việc quan trọng nhất trong thuật toán gradient descent là đi tìm đạo hàm của các hệ số đối với loss function.
 - Giờ chúng ta sẽ tính đạo hàm của các hệ số trong neural network với thuật toán backpropagation.

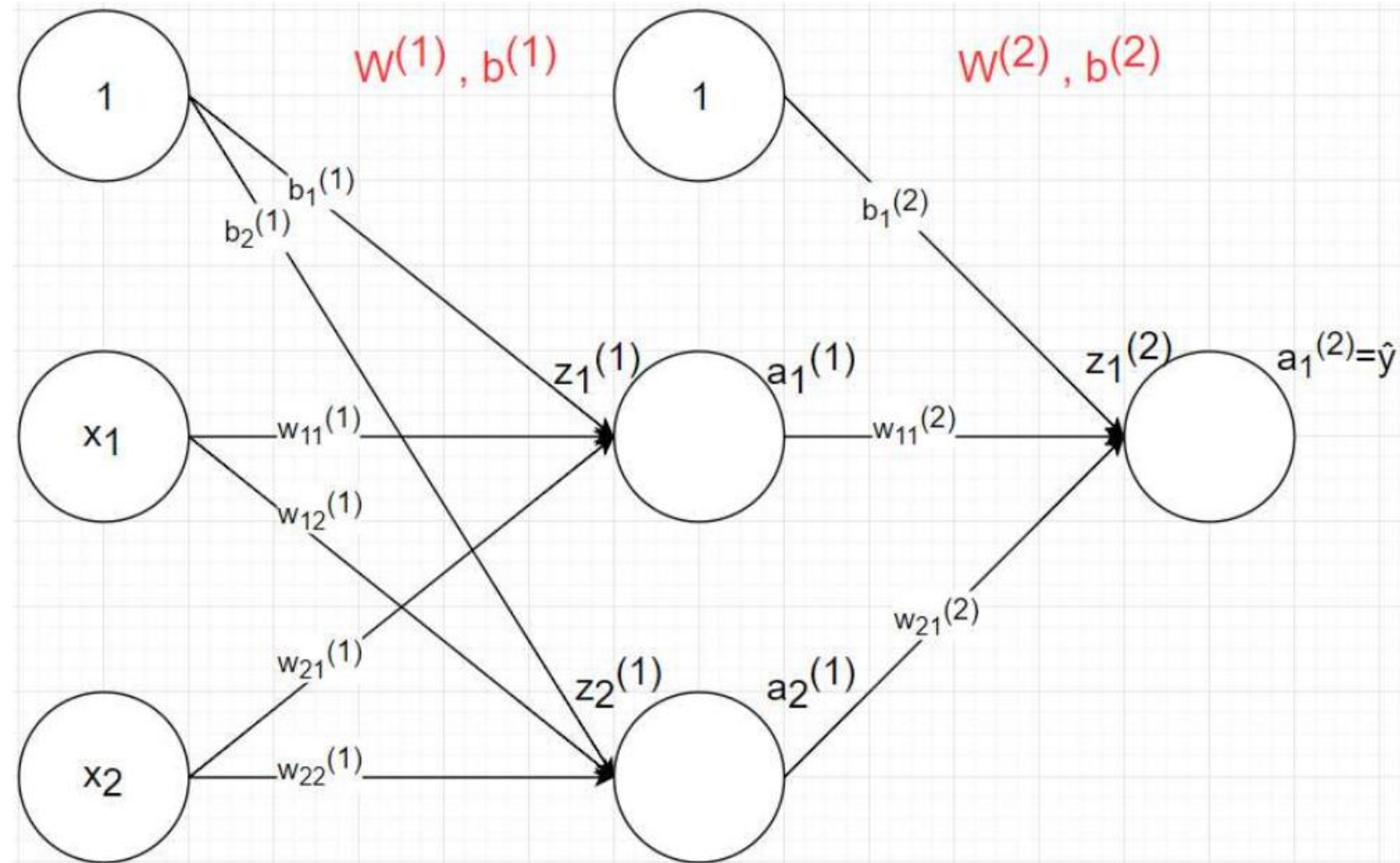
Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Bài trước đã chứng minh là mô hình logistic regression không thể biểu diễn được toán tử XOR.
 - Để biểu diễn toán tử XOR ta cần thêm 1 hidden layer với 2 node.
 - Bảng chân lý cho toán tử XOR

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Mô hình



Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Nhắc lại kiến thức
 - Mô hình trên có 2 layer (số lượng layer của mô hình không tính input layer)
 - Mô hình: 2-2-1, nghĩa là 2 node trong input layer, 1 hidden layer có 2 node và output layer có 1 node.
 - Input layer và hidden layer luôn thêm node 1 để tính bias cho layer sau, nhưng không tính vào số lượng node trong layer
 - Ở mỗi node trong hidden layer và output layer đều thực hiện 2 bước: tính tổng linear và áp dụng activation function.
 - Các hệ số và bias tương ứng được ký hiệu như trong hình ở slide trước

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Nhắc lại kiến thức

Feedforward

- $z_1^{(1)} = b_1^{(1)} + x_1 * w_{11}^{(1)} + x_2 * w_{21}^{(1)}$
- $a_1^{(1)} = \sigma(z_1^{(1)})$
- $z_2^{(1)} = b_2^{(1)} + x_1 * w_{12}^{(1)} + x_2 * w_{22}^{(1)}$
- $a_2^{(1)} = \sigma(z_2^{(1)})$
- $z_1^{(2)} = b_1^{(2)} + a_1^{(1)} * w_{11}^{(2)} + a_2^{(1)} * w_{21}^{(2)}$
- $\hat{y} = a_1^{(2)} = \sigma(z_1^{(2)})$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Nhắc lại kiến thức
 - Dạng ma trận:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, Y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

$$Z^{(1)} = X * W^{(1)} + b^{(1)}$$

$$A^{(1)} = \sigma(Z^{(1)})$$

$$Z^{(2)} = A^{(1)} * W^{(2)} + b^{(2)}$$

$$\hat{Y} = A^{(2)} = \sigma(Z^{(2)})$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Loss function: Hàm loss function vẫn dùng giống như trong bài trước
 - Với mỗi điểm $(x^{(i)}, y_i)$, gọi hàm loss function

$$L = -(y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))$$

- Hàm loss function trên toàn bộ dữ liệu

$$J = -\frac{1}{N} * \sum_{i=1}^N (y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent
 - Để áp dụng gradient descent ta cần tính được đạo hàm của các hệ số W và bias b với hàm loss function.
 - *** Kí hiệu chuẩn về đạo hàm:**
 - Khi hàm $f(x)$ là hàm 1 biến x , ví dụ: $f(x) = 2 * x + 1$. Đạo hàm của f đối với biến x kí hiệu là $\frac{df}{dx}$
 - Khi hàm $f(x, y)$ là hàm nhiều biến, ví dụ $(x,y) = x^2 + y^2$. Đạo hàm f với biến x kí hiệu là $\frac{\partial f}{\partial x}$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent
 - Với mỗi điểm $(x^{[i]}, y_i)$, gọi hàm loss function

$L = -(y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))$, trong đó

$\hat{y}_i = a_1^{(2)} = \sigma(a_1^{(1)} * w_{11}^{(2)} + a_2^{(1)} * w_{21}^{(2)} + b_1^{(2)})$ là giá trị model dự đoán, y_i là giá trị thật của dữ liệu.

$$\frac{\partial L}{\partial \hat{y}_i} = -\frac{\partial (y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))}{\partial \hat{y}_i} = -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{(1 - \hat{y}_i)}\right)$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network

- Gradient descent

- Tính đạo hàm L với $W^{(2)}$, $b^{(2)}$

- Áp dụng chain rule ta có:

$$\frac{\partial L}{\partial b_1^{(2)}} = \frac{dL}{dy_i} * \frac{\partial \hat{y}_i}{\partial b_1^{(2)}}$$

$$\frac{\partial \hat{y}_i}{\partial b_1^{(2)}} = \hat{y}_i * (1 - \hat{y}_i)$$

$$\frac{\partial \hat{y}_i}{\partial w_{11}^{(2)}} = a_1^{(1)} * \hat{y}_i * (1 - \hat{y}_i)$$

$$\frac{\partial \hat{y}_i}{\partial w_{21}^{(2)}} = a_2^{(1)} * \hat{y}_i * (1 - \hat{y}_i)$$

$$\frac{\partial \hat{y}_i}{\partial a_1^{(1)}} = w_{11}^{(2)} * \hat{y}_i * (1 - \hat{y}_i)$$

$$\frac{\partial \hat{y}_i}{\partial a_2^{(1)}} = w_{21}^{(2)} * \hat{y}_i * (1 - \hat{y}_i)$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent

Do đó

$$\frac{\partial L}{\partial b_1^{(2)}} = \frac{\partial L}{\partial \hat{y}_i} * \frac{\partial \hat{y}_i}{\partial b_1^{(2)}} = -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{(1 - \hat{y}_i)}\right) * \hat{y}_i * (1 - \hat{y}_i) = -(y_i * (1 - \hat{y}_i) - (1 - y_i) * \hat{y}_i) = \hat{y}_i - y_i$$

Tương tự

$$\frac{\partial L}{\partial w_{11}^{(2)}} = a_1^{(1)} * (\hat{y}_i - y_i)$$

$$\frac{\partial L}{\partial w_{21}^{(2)}} = a_2^{(1)} * (\hat{y}_i - y_i)$$

$$\frac{\partial L}{\partial a_1^{(1)}} = w_{11}^{(2)} * (\hat{y}_i - y_i)$$

$$\frac{\partial L}{\partial a_2^{(1)}} = w_{21}^{(2)} * (\hat{y}_i - y_i)$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent
 - Biểu diễn dưới dạng ma trận

$$\frac{\partial L}{\partial W} = \begin{bmatrix} \frac{\partial L}{\partial w_{11}} & \cdots & \frac{\partial L}{\partial w_{1n}} \\ \frac{\partial L}{\partial w_{21}} & \cdots & \frac{\partial L}{\partial w_{2n}} \\ \vdots & & \vdots \\ \frac{\partial L}{\partial w_{m1}} & \cdots & \frac{\partial L}{\partial w_{mn}} \end{bmatrix}$$

- Do đó:

$$\frac{\partial J}{\partial W^{(2)}} = (A^{(1)})^T * (\hat{Y} - Y), \frac{\partial J}{\partial b^{(2)}} = (\text{sum}(\hat{Y} - Y))^T, \frac{\partial J}{\partial A^{(1)}} = (\hat{Y} - Y) * (W^{(2)})^T$$

phép tính sum tính tổng các cột của ma trận

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent
 - Biểu diễn dưới dạng ma trận

$$\frac{\partial L}{\partial W} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{21} \\ w_{31} & w_{32} \end{bmatrix} \Rightarrow \text{sum}(W) = (w_{11} + w_{21} + w_{31}, w_{12} + w_{22} + w_{32})$$
$$(\text{sum}(W))^T = \begin{bmatrix} w_{11} + w_{21} + w_{31} \\ w_{12} + w_{22} + w_{32} \end{bmatrix}$$

- Như vậy đã tính xong đạo hàm của L với hệ số $W^{(2)}, b^{(2)}$.
- Giờ cần tính đạo hàm của L với hệ số $W^{(1)}, b^{(1)}$.

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent

- Tính đạo hàm L với $W^{(1)}, b^{(1)}$

$$\text{Do } a_1^{(1)} = \sigma(b_1^{(1)} + x_1 * w_{11}^{(1)} + x_2 * w_{21}^{(1)})$$

$$\text{Áp dụng chain rule ta có: } \frac{\partial L}{\partial b_1^{(1)}} = \frac{\partial L}{\partial a_1^{(1)}} * \frac{\partial a_1^{(1)}}{\partial b_1^{(1)}}$$

Ta có:

$$\frac{\partial a_1^{(1)}}{\partial b_1^{(1)}} = \frac{\partial a_1^{(1)}}{z_1^{(1)}} * \frac{z_1^{(1)}}{\partial b_1^{(1)}} = a_1^{(1)} * (1 - a_1^{(1)})$$

Do đó

$$\frac{\partial L}{\partial b_1^{(1)}} = a_1^{(1)} * (1 - a_1^{(1)}) * w_{11}^{(2)} * (\hat{y}_i - y_i)$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent
 - Tính đạo hàm L với $W^{(1)}, b^{(1)}$

Tương tự

$$\frac{\partial L}{\partial w_{11}^{(1)}} = x_1 * a_1^{(1)} * (1 - a_1^{(1)}) * w_{11}^{(2)} * (\hat{y}_i - y_i)$$

$$\frac{\partial L}{\partial w_{12}^{(1)}} = x_1 * a_2^{(1)} * (1 - a_2^{(1)}) * w_{11}^{(2)} * (\hat{y}_i - y_i)$$

$$\frac{\partial L}{\partial w_{21}^{(1)}} = x_2 * a_1^{(1)} * (1 - a_1^{(1)}) * w_{21}^{(2)} * (\hat{y}_i - y_i)$$

$$\frac{\partial L}{\partial w_{22}^{(1)}} = x_2 * a_2^{(1)} * (1 - a_2^{(1)}) * w_{21}^{(2)} * (\hat{y}_i - y_i)$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent
 - Viết dưới dạng ma trận

Có thể tạm viết dưới dạng chain rule là $\frac{\partial J}{\partial W^{(1)}} = \frac{\partial J}{\partial A^{(1)}} * \frac{\partial A^{(1)}}{\partial Z^{(1)}} * \frac{\partial Z^{(1)}}{\partial W^{(1)}} \quad (1).$

Từ trên đã tính được $\frac{\partial J}{\partial A^{(1)}} = (\hat{Y} - Y) * (W^{(2)})^T$

Đạo hàm của hàm sigmoid $\frac{d\sigma(x)}{dx} = \sigma(x) * (1 - \sigma(x))$ và $A^{(1)} = \sigma(Z^{(1)})$

nên trong (1) có thể hiểu là:

$$\frac{\partial A^{(1)}}{\partial Z^{(1)}} = A^{(1)} * (1 - A^{(1)})$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent
 - Viết dưới dạng ma trận

Cuối cùng, $Z^{(1)} = X * W^{(1)} + b^{(1)}$ nên có thể tạm hiểu $\frac{\partial Z^{(1)}}{\partial W^{(1)}} = X$

nó giống như $f(x) = a * x + b \Rightarrow \frac{df}{dx} = a$

Kết hợp tất cả lại $\frac{\partial J}{\partial W^{(1)}} = X^T * (((\hat{Y} - Y) * (W^{(2)})^T) \otimes A^{(1)} \otimes (1 - A^{(1)}))$

Thế khi nào thì dùng element-wise ($\otimes$), khi nào dùng nhân ma trận ($*$)?

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network
 - Gradient descent
 - Khi tính đạo hàm ngược lại qua bước activation thì dùng ($\otimes$).
 - Khi có phép tính nhân ma trận thì dùng ($*$), nhưng đặc biệt chú ý đến kích thước ma trận và dùng transpose nếu cần thiết.
Ví dụ: ma trận X kích thước $N*3$, W kích thước $3*4$, $Z = X * W$ sẽ có kích thước $N*4$ thì:

$$\frac{\partial J}{\partial W} = X^T * \left(\frac{\partial J}{\partial Z}\right) \text{ và } \frac{\partial J}{\partial X} = \left(\frac{\partial J}{\partial Z}\right) * W^T$$

Mô hình mạng nơ-ron (neural network)

- Bài toán XOR với neural network

- Gradient descent

- Tính đạo hàm L với $W^{(1)}$, $b^{(1)}$

Tương tự,
$$\frac{\partial L}{\partial b^{(1)}} = \text{sum}(((\hat{Y} - Y) * (W^{(2)})^T) \otimes A^{(1)})^T$$

- Như vậy, ta đã tính xong hết đạo hàm của loss function với các hệ số W và bias b , giờ có thể áp dụng gradient descent để giải bài toán.

Mô hình mạng nơ-ron (neural network)

- Mô hình tổng quát

1. Bước 1: Tính $\frac{\partial J}{\partial \hat{Y}}$, trong đó $\hat{Y} = A^{(3)}$

2. Bước 2: Tính $\frac{\partial J}{\partial \hat{W}^{(3)}} = (A^{(2)})^T * (\frac{\partial J}{\partial \hat{Y}} \otimes \frac{\partial A^{(3)}}{\partial Z^{(3)}})$, $\frac{\partial J}{\partial \hat{b}^{(3)}} = (sum(\frac{\partial J}{\partial \hat{Y}} \otimes \frac{\partial A^{(3)}}{\partial Z^{(3)}}))^T$ và tính $\frac{\partial J}{\partial A^{(2)}} = (\frac{\partial J}{\partial \hat{Y}} \otimes \frac{\partial A^{(3)}}{\partial Z^{(3)}}) * (W^{(3)})^T$

3. Bước 3: Tính $\frac{\partial J}{\partial \hat{W}^{(2)}} = (A^{(1)})^T * (\frac{\partial J}{\partial A^{(2)}} \otimes \frac{\partial A^{(2)}}{\partial Z^{(2)}})$, $\frac{\partial J}{\partial \hat{b}^{(2)}} = (sum(\frac{\partial J}{\partial A^{(2)}} \otimes \frac{\partial A^{(2)}}{\partial Z^{(2)}}))^T$ và tính $\frac{\partial J}{\partial A^{(1)}} = (\frac{\partial J}{\partial A^{(2)}} \otimes \frac{\partial A^{(2)}}{\partial Z^{(2)}}) * (W^{(2)})^T$

4. Bước 4: Tính $\frac{\partial J}{\partial \hat{W}^{(1)}} = (A^{(0)})^T * (\frac{\partial J}{\partial A^{(1)}} \otimes \frac{\partial A^{(1)}}{\partial Z^{(1)}})$, $\frac{\partial J}{\partial \hat{b}^{(1)}} = (sum(\frac{\partial J}{\partial A^{(1)}} \otimes \frac{\partial A^{(1)}}{\partial Z^{(1)}}))^T$, trong đó $A^{(0)} = X$

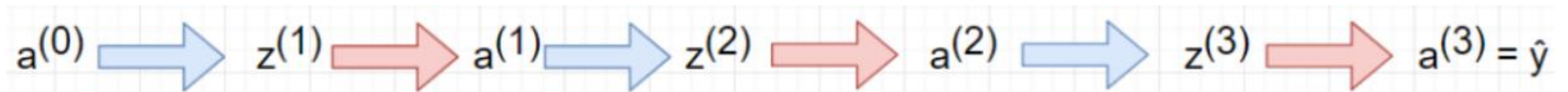
Mô hình mạng nơ-ron (neural network)

- Mô hình tổng quát
 - Nếu network có nhiều layer hơn thì cứ tiếp tục cho đến khi tính được đạo hàm của loss function J với tất cả các hệ số W và bias b .
 - Nếu activation function là sigmoid thì:

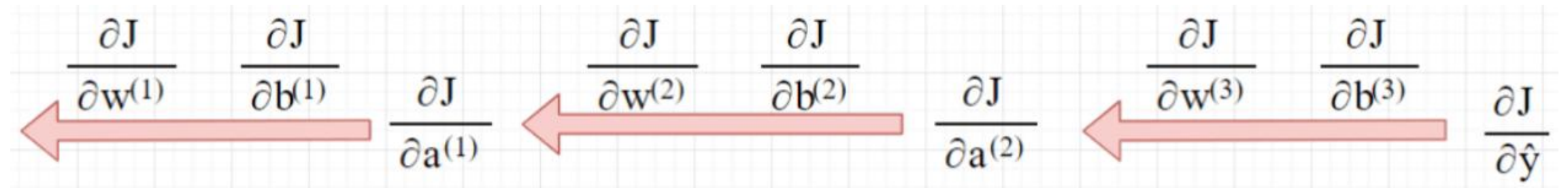
$$\frac{\partial A^{(i)}}{\partial Z^{(i)}} = A^{(i)} \otimes (1 - A^{(i)})$$

Mô hình mạng nơ-ron (neural network)

- Mô hình tổng quát
 - Ở bài trước quá trình feedforward như sau:



- Ở bài này, quá trình tính đạo hàm ngược lại:



➔ Vì vậy thuật toán được gọi là backpropagation (lan truyền ngược)

Tiếp cận mạng nơ-ron

- **Các hàm kích hoạt thường được sử dụng:**

- Hàm sigmoid (hàm logistic):

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Hàm hyperbol:

$$h(x) = \frac{1 - e^{-x}}{1 + e^{-x}}$$

- Hàm tang-hyperbol:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tiếp cận mạng nơ-ron

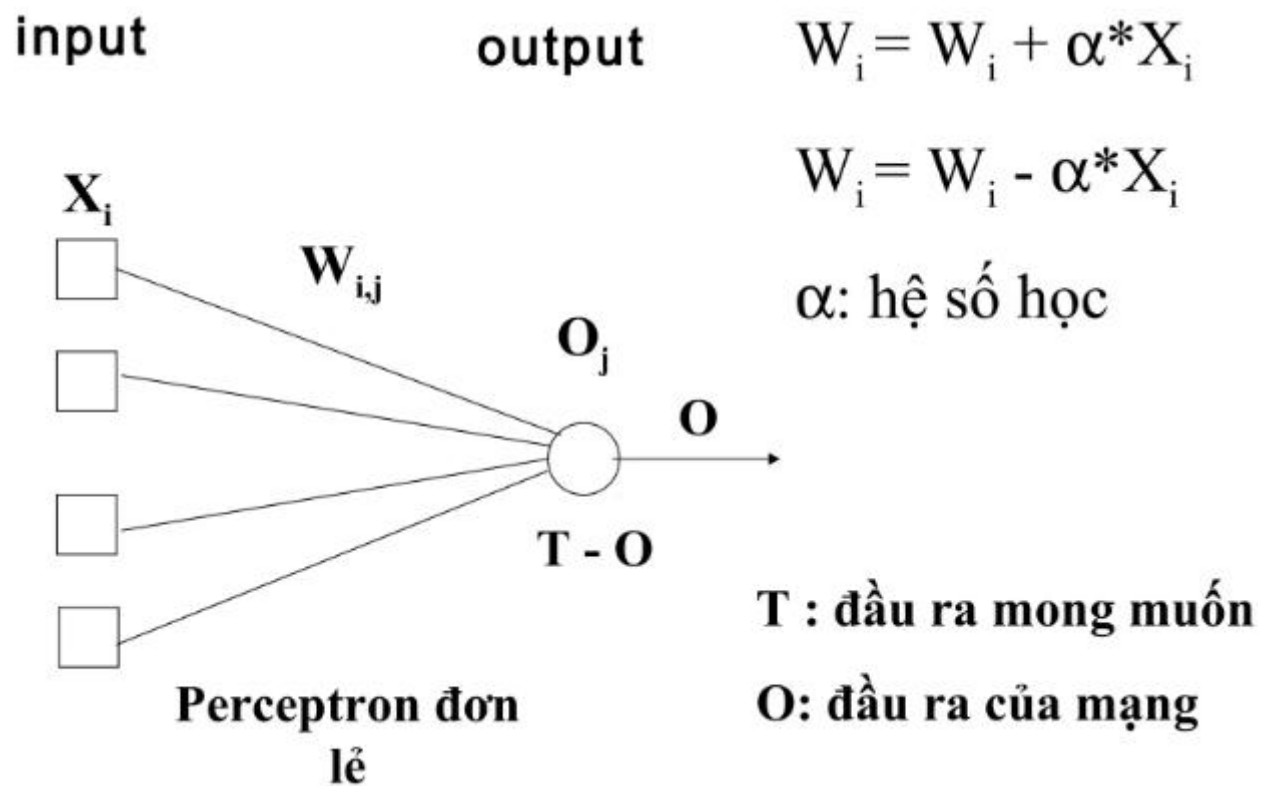
- **Mạng Perceptron (cho bài toán phân lớp nhị phân):**
 - Perceptron là một trong những thuật toán học máy đầu tiên và đơn giản nhất, được Frank Rosenblatt phát minh vào năm 1957.
 - Perceptron được thiết kế như một mô hình phân loại nhị phân, nghĩa là dự đoán một trong hai lớp có thể xác định.
 - Mặc dù đơn giản, Perceptron là nguồn cảm hứng cho các mạng neuron sâu ngày nay.

Tiếp cận mạng nơ-ron

- **Mạng Perceptron (cho bài toán phân lớp nhị phân):**
 - Một Perceptron có thể được xem như là một neuron nhân tạo đơn lẻ. Nó bao gồm:
 - Đầu vào (inputs): Mỗi đầu vào được đánh trọng số (weights) riêng biệt.
 - Tổng trọng số (weighted sum): Tính tổng các đầu vào nhân với trọng số tương ứng.
 - Hàm kích hoạt (activation function): Đơn vị tính toán sẽ quyết định neuron có nên được kích hoạt hay không.
 - Thông thường, hàm kích hoạt được sử dụng cho Perceptron là hàm step function, nghĩa là nếu tổng trọng số vượt quá một ngưỡng nào đó, đầu ra sẽ là 1 (hoặc lớp A), còn nếu không vượt quá, đầu ra sẽ là -1 (hoặc lớp B).

Tiếp cận mạng nơ-ron

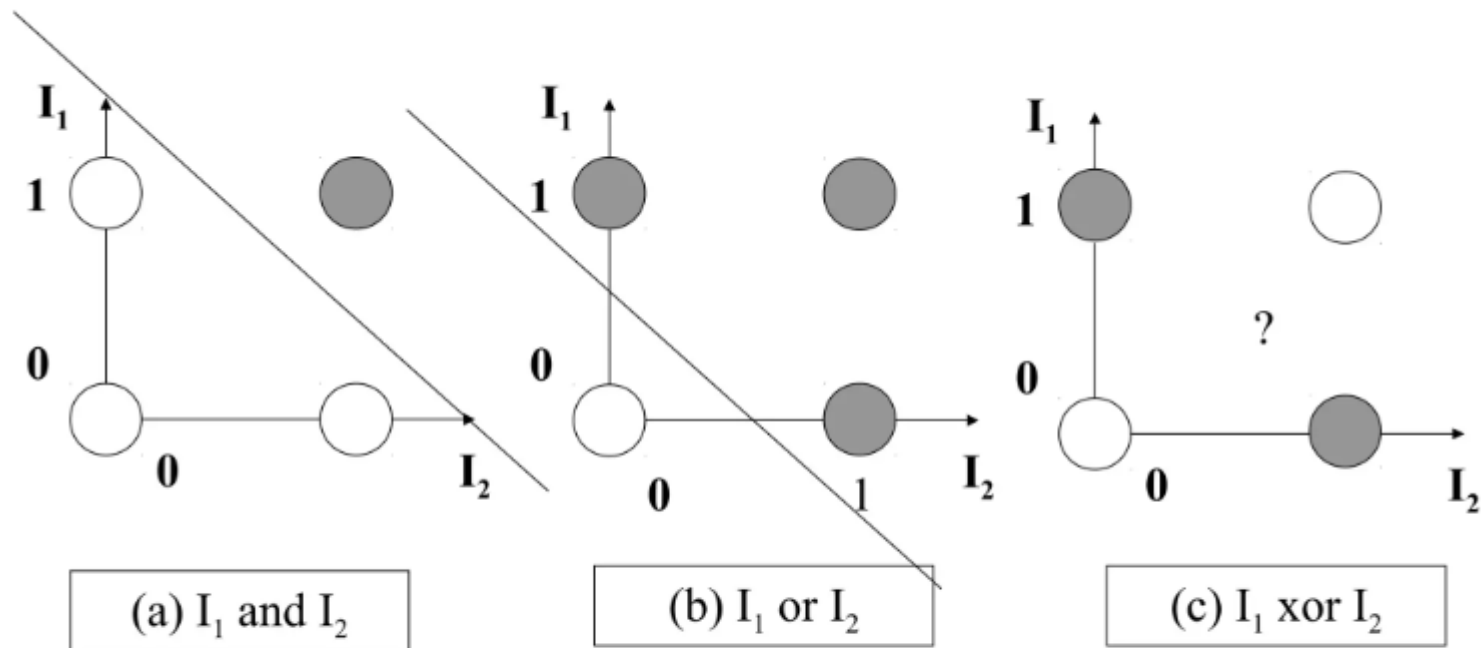
- **Mạng Perceptron (cho bài toán phân lớp nhị phân):**



Tiếp cận mạng nơ-ron

- **Mạng Perceptron:**

Khả năng phân chia tuyến tính trong mạng Perceptron



$$Y(I_1, I_2) = \begin{cases} 1 & \text{nếu } W_1 I_1 + W_2 I_2 > \theta \\ 0 & \text{ngược lại} \end{cases}$$

Tiếp cận mạng nơ-ron

- **Mạng Perceptron:**

Hạn chế của mạng Perceptron

- Năm 1969, Minsky và Papert đã phân tích và chứng minh sự thất bại của mạng Perceptron với bài toán **XOR**.

I_1	I_2	$y(I_1, I_2) = I_1 \text{ XOR } I_2$
1	1	0
1	0	1
0	1	1
0	0	0

Tiếp cận mạng nơ-ron

- **Mạng Perceptron:**

Xét từng trường hợp:

Nếu $[I_1, I_2]=[0,0]$ thì $y(I_1, I_2)=0 \Rightarrow W_1*0 + W_2*0 < \theta \Rightarrow \theta > 0$

Nếu $[I_1, I_2]=[0,1]$ thì $y(I_1, I_2)=1 \Rightarrow W_1*0 + W_2*1 \geq \theta \Rightarrow \theta \leq W_1$

Nếu $[I_1, I_2]=[1,0]$ thì $y(I_1, I_2)=1 \Rightarrow W_1*1 + W_2*0 \geq \theta \Rightarrow \theta \leq W_2$

Nếu $[I_1, I_2]=[1,1]$ thì $y(I_1, I_2)=0 \Rightarrow W_1*1 + W_2*1 < \theta \Rightarrow \theta > W_1 + W_2$

$\Rightarrow$ Rõ ràng không thể tìm được W_1, W_2, θ thỏa mãn 4 bất đẳng thức trên.

$\Rightarrow$ Mô hình Perceptron không thể tìm ra lời giải trong trường hợp này vì hàm **XOR** không phải là hàm ngưỡng tuyến tính.

Tiếp cận mạng nơ-ron

- **Mạng Perceptron:**

Ví dụ minh họa việc học của mạng Perceptron:

	Cột thêm vào	Độ dài đài hoa	Độ rộng đài hoa	Độ dài cánh hoa	Độ rộng cánh hoa	Loài
Cây Iris A	1	4.7	3.2	1.3	0.2	-1
Cây Iris B	1	6.1	2.8	4.7	1.2	1
Cây Iris C	1	5.6	3.0	4.1	1.3	1
Cây Iris D	1	5.8	2.7	5.1	1.9	-1
Cây Iris E	1	6.5	3.2	5.1	2.0	-1
Cây Iris Q	1	5.8	2.7	3.9	1.2	???

Khởi tạo trọng số: $w_1=1$; $w_2=0$; $w_3=0$; $w_4=0$; $w_5=1$

Tiếp cận mạng nơ-ron

- **Mạng Perceptron:**

❖ Dự đoán lần thứ nhất đối với mẫu “Cây Iris A”

$$\sum_{i=1}^5 w_i x_i = 1 * 1 + 0 * 4.7 + 0 * 3.2 + 0 * 1.3 + 1 * 0.2 = 1.2$$

Perceptron dự đoán “**Loài**” có kết quả là **1** so với kết quả đúng là **-1**

❖ Cập nhật trọng số mạng

$w_i \leftarrow w_i - \alpha * x_i$ Nếu kết quả “**Loài**” là **1** so với kết quả đúng là **-1**

(α : Tỷ lệ học nên chọn nhỏ, ở đây ta chọn $\alpha = 0.05$)

Tiếp cận mạng nơ-ron

- **Mạng Perceptron:**

$$w_1 \leftarrow w_1 - 0.05 * x_1 = 1 - 0.05 * 1 = 0.95$$

$$w_2 \leftarrow w_2 - 0.05 * x_2 = 0 - 0.05 * 4.7 = -0.24$$

$$w_3 \leftarrow w_3 - 0.05 * x_3 = 0 - 0.05 * 3.2 = -0.16$$

$$w_4 \leftarrow w_4 - 0.05 * x_4 = 0 - 0.05 * 1.3 = -0.07$$

$$w_5 \leftarrow w_5 - 0.05 * x_5 = 1 - 0.05 * 0.2 = 0.99$$

Các trọng số này dùng để luyện cho những mẫu kế tiếp

❖ **Điều kiện dừng:**

Tất cả các mẫu đã được dự đoán đúng.

Dừng sau một số bước lặp do người dùng quyết định.

Tiếp cận mạng nơ-ron

- **Mạng Perceptron:**

Nhận xét: Cập nhật trọng số sao cho tối thiểu sai lệch.

(1) $w_i \leftarrow w_i - \alpha * x_i$ Nếu kết quả “**Loài**” là 1 so với kết quả đúng là -1

(2) $w_i \leftarrow w_i + \alpha * x_i$ Nếu kết quả “**Loài**” là -1 so với kết quả đúng 1

(1) $w_i x_i$ góp phần làm cho $\sum_{i=1}^n w_i x_i > 0$.

- Trọng số mới là $w_i \leftarrow w_i - \alpha * x_i$. Vì thế bước kế tiếp cho cùng mẫu dữ liệu ta được $(w_i - \alpha * x_i) * x_i = w_i x_i - \alpha * x_i^2 < w_i x_i$

- Như vậy $\sum_{i=1}^n w_i x_i$ sau khi thay đổi trọng số nhỏ hơn trước đó.

- Vì vậy kết quả dự đoán sẽ gần về kết quả đúng là -1.

Giới thiệu mạng nơ-ron

- **Mạng Perceptron:**

Nguyên lý hoạt động của mạng Perceptron

Bước 1: Xác định giá trị của ***Err***

$$Err = T - O \text{ (} T : \text{đầu ra mong muốn)}$$

if ***Err*** > 0 then tăng ***O***

else giảm ***O***

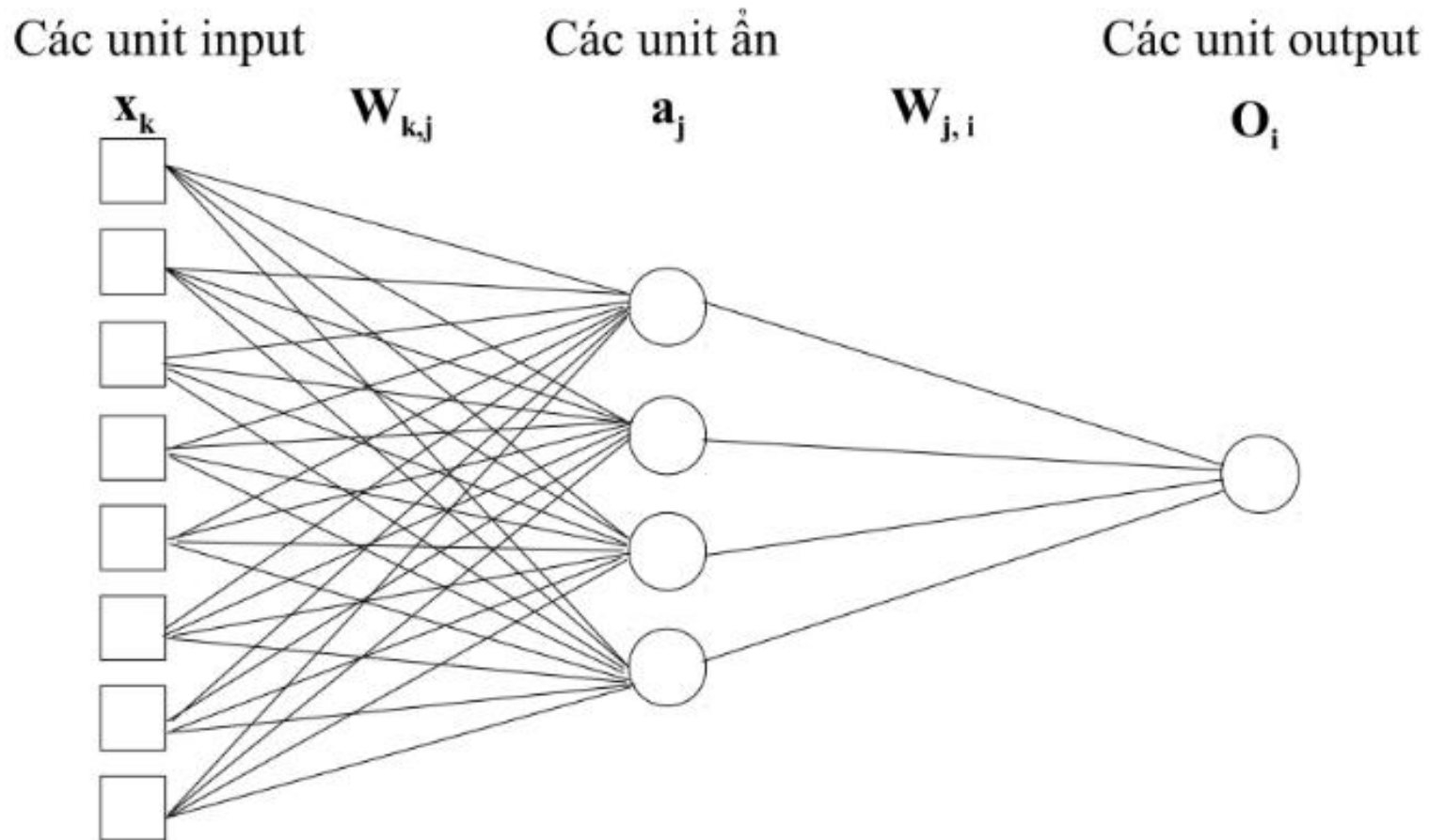
Bước 2: Cập nhật giá trị ma trận trọng số

$$W_j = W_j + \alpha * x_j * Err. \text{ (}\alpha \text{ là hệ số học)}$$

Bước 3: Lặp lại bước 1 và 2 cho đến khi ***Err*** < ϵ thì dừng.

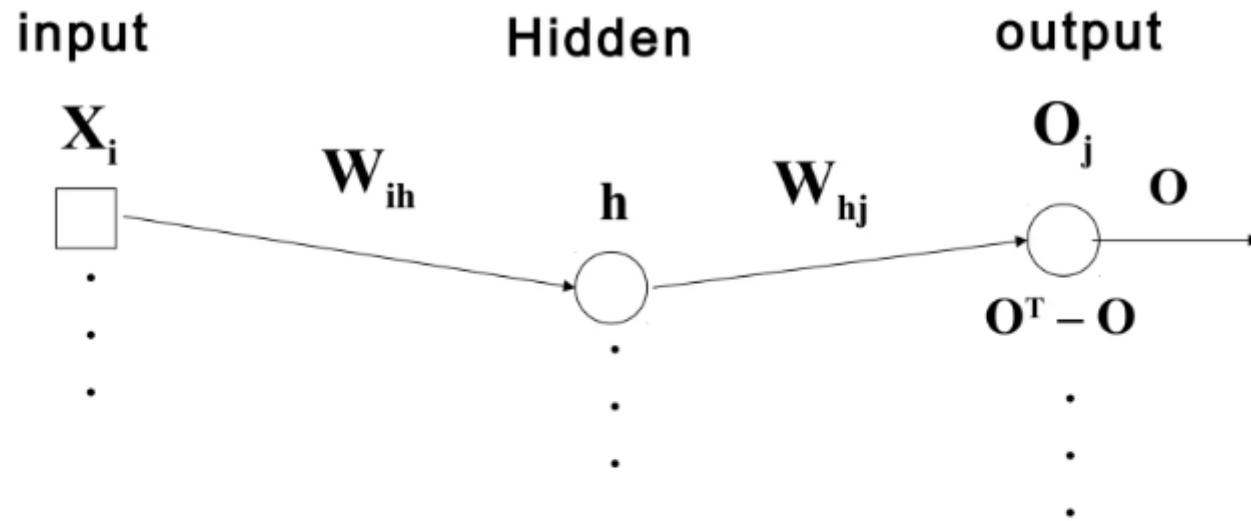
Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**



Tiếp cận mạng nơ-ron

- Mạng lan truyền ngược (back propagation):



$$\delta_o = o(1-o)(T-o)$$

$$W_{hj} = W_{hj} + \alpha * \delta_{oj} * o_j$$

$$W_{ih} = W_{ih} + \alpha * \delta_o * W_h * o_h * (1-o_h) * x_i$$

Trong đó: o là đầu ra thật, T là đầu ra mong muốn, δ_o là sai lệch của nút o

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

Tập mẫu cho quá trình học

Mẫu	Đầu vào	Đầu ra mong muốn
1	$x^1 = (x_1^1, x_2^1, \dots, x_n^1)$	y^1
2	$x^2 = (x_1^2, x_2^2, \dots, x_n^2)$	y^2
...	...	...
M	$x^m = (x_1^m, x_2^m, \dots, x_n^m)$	y^m

❖ Cho mẫu x^k vào luyện -> Cho ra O^k sai lệch với kết quả mong muốn y^k . Sai lệch của mẫu thứ k là E_k :

$$E_k = \frac{1}{2} * (y^k - O^k)^2 = \frac{1}{2} (y^k - f(w^T x))^2$$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

- ❖ Sai lệch của tất cả các mẫu:

$$E = \sum_{k=1}^m E_k = E_1 + E_2 + \dots + E_m$$

- ❖ Cập nhật trọng số: theo hướng cực tiểu hóa sai lệch

$$w = w - \alpha * E'_k(w)$$

α : là hệ số học

$$E'_k(w) = \frac{d}{dw} \left(\frac{1}{2} (y^k - f(w^T x))^2 \right) = -(y^k - f(w^T x)) f'(w^T x)$$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

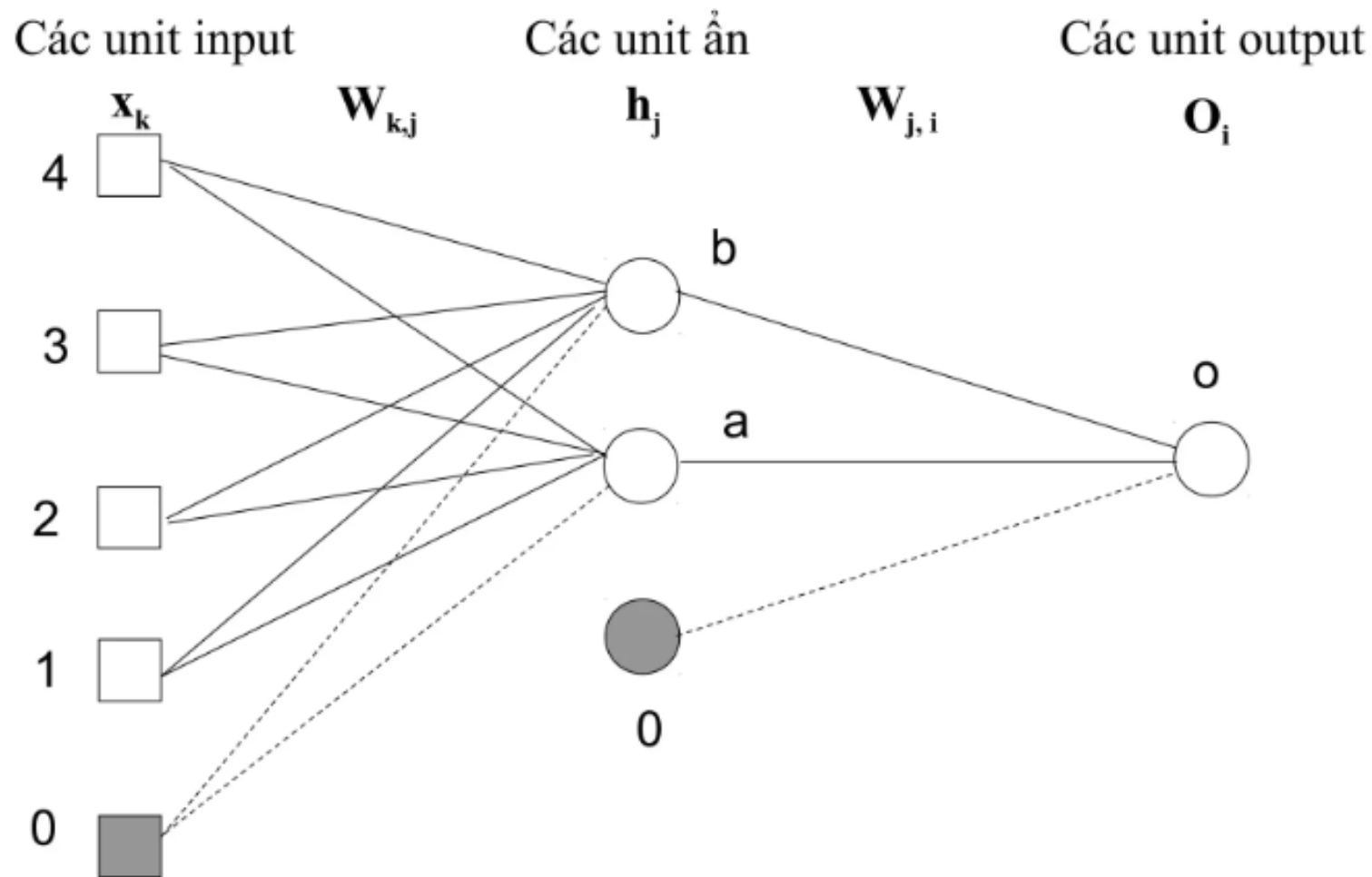
Ví dụ minh họa cho phương pháp lan truyền ngược

	Cột thêm vào	Độ dài đài hoa	Độ rộng đài hoa	Độ dài cánh hoa	Độ rộng cánh hoa	Loài
Cây Iris A	1	4.7	3.2	1.3	0.2	0
Cây Iris B	1	6.1	2.8	4.7	1.2	1
Cây Iris C	1	5.6	3.0	4.1	1.3	1
Cây Iris D	1	5.8	2.7	5.1	1.9	0
Cây Iris E	1	6.5	3.2	5.1	2.0	0
Cây Iris Q	1	5.8	2.7	3.9	1.2	???

Chọn hàm truyền: $f(x) = \frac{1}{1 + e^{-x}}$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**



Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

Khởi tạo các trọng số có giá trị ngẫu nhiên nhỏ

$W_{0a} = 0.0$ Trọng số cột giá trị -1 trong đơn vị ẩn a

$W_{1a} = 0.1$ Trọng số trong đơn vị ẩn a từ giá trị nhập thứ 1

$W_{2a} = -0.1$ Trọng số trong đơn vị ẩn a từ giá trị nhập thứ 2

$W_{3a} = -0.1$ Trọng số trong đơn vị ẩn a từ giá trị nhập thứ 3

$W_{4a} = 0.0$ Trọng số trong đơn vị ẩn a từ giá trị nhập thứ 4

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

$W_{0b} = 0.0$ Trọng số cột giá trị -1 trong đơn vị ẩn b

$W_{1b} = -0.1$ Trọng số trong đơn vị ẩn b từ giá trị nhập thứ 1

$W_{2b} = 0.2$ Trọng số trong đơn vị ẩn b từ giá trị nhập thứ 2

$W_{3b} = 0.1$ Trọng số trong đơn vị ẩn b từ giá trị nhập thứ 3

$W_{4b} = -0.1$ Trọng số trong đơn vị ẩn b từ giá trị nhập thứ 4

$W_{0o} = 0.1$ Trọng số cột giá trị -1 trong đơn vị xuất o

$W_{ao} = 0.2$ Trọng số cột trong đơn vị ẩn b từ đơn vị ẩn a

$W_{bo} = -0.1$ Trọng số cột trong đơn vị ẩn b từ đơn vị ẩn b

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

Bước 1: Dự đoán

➤ Đối với “cây Iris A” đầu tiên ta tính giá trị xuất của **nút ẩn a**

$$\begin{aligned}o_a &= f(w_{0a} + w_{1a}x_1 + w_{2a}x_2 + w_{3a}x_3 + w_{4a}x_4) \\&= f(0.0 + 0.1*4.7 + (-0.1)*3.2 + (-0.1)*1.3 + 0.0*0.2) \\&= f(0.02) = 1/(1 + e^{-0.02}) = 0.5050\end{aligned}$$

➤ Tương tự với **nút ẩn b**

$$\begin{aligned}o_b &= f(w_{0b} + w_{1b}x_1 + w_{2b}x_2 + w_{3b}x_3 + w_{4b}x_4) \\&= f(0.0 + (-0.1)*4.7 + 0.2*3.2 + 0.1*1.3 + (-0.1)*0.2) \\&= f(0.28) = 1/(1 + e^{-0.28}) = 0.5695\end{aligned}$$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

➤ Cuối cùng ta tính giá trị xuất cho **nút xuất o**

$$\begin{aligned}o_o &= f(w_{oo} + w_{ao}o_a + w_{bo}o_b) \\&= f(0.1 + 0.2*0.5050 + (-0.1)*0.5695) \\&= f(0.1440) = 1/(1 + e^{-0.1440}) = 0.5359\end{aligned}$$

⇒ Giá trị dự đoán cho “cây Iris A” là 0.5359

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

Bước 2: Xác định sai lệch

➤ Dùng kỹ thuật lan truyền ngược (back propagation)

Với t_o : giá trị xuất mong muốn

o_o : giá trị xuất thật

Sai lệch cần tính cho nút xuất o : $\delta_o = o_o(1 - o_o)(t_o - o_o)$

Sai lệch cần tính cho nút ẩn h từ o là: $o_h(1-o_h)w_{ho} \delta_o$

➤ Trong ví dụ đang xét “cây Iris A” có $t_o = 0$.

Sai lệch của **nút xuất o** :

$$\delta_o = o_o(1-o_o)(t_o-o_o) = 0.5359*(1-0.5359)*(0-0.5359) = -0.1333$$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

Sai lệch của **nút ẩn a**:

$$\begin{aligned}\delta_a &= o_a(1-o_a)w_{ao}\delta_o = 0.5050*(1-0.5050)*0.2*(-0.1333) \\ &= -0.0067\end{aligned}$$

Sai lệch của **nút ẩn b**:

$$\begin{aligned}\delta_b &= o_b(1-o_b)w_{bo}\delta_o = 0.5695*(1-0.5695)*(-0.1)*(-0.1333) \\ &= 0.0032\end{aligned}$$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

Bước 3: Bước cập nhật trọng số

Chọn tỉ lệ học $\alpha = 0.1$

$$w_{0a} = w_{0a} + \alpha * \delta_a * 1 = 0.0 + 0.1 * (-0.0067) * 1 = -0.0007$$

$$w_{1a} = w_{1a} + \alpha * \delta_a * x_1 = 0.1 + 0.1 * (-0.0067) * 4.7 = 0.0969$$

$$w_{2a} = w_{2a} + \alpha * \delta_a * x_2 = -0.1 + 0.1 * (-0.0067) * 3.2 = -0.1021$$

$$w_{3a} = w_{3a} + \alpha * \delta_a * x_3 = -0.1 + 0.1 * (-0.0067) * 1.2 = -0.1009$$

$$w_{4a} = w_{4a} + \alpha * \delta_a * x_4 = 0.0 + 0.1 * (-0.0067) * 0.2 = -0.0001$$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

$$w_{0b} = w_{0b} + \alpha * \delta_b * 1 = 0.0 + 0.1 * (0.0032) * 1 = 0.0003$$

$$w_{1b} = w_{1b} + \alpha * \delta_b * x_1 = -0.1 + 0.1 * (0.0032) * 4.7 = -0.0985$$

$$w_{2b} = w_{2b} + \alpha * \delta_b * x_2 = 0.2 + 0.1 * (0.0032) * 3.2 = 0.2010$$

$$w_{3b} = w_{3b} + \alpha * \delta_b * x_3 = 0.1 + 0.1 * (0.0032) * 1.2 = 0.1004$$

$$w_{4b} = w_{4b} + \alpha * \delta_b * x_4 = -0.1 + 0.1 * (0.0032) * 0.2 = -0.0999$$

$$w_{0o} = w_{0o} + \alpha * \delta_o * 1 = 0.1 + 0.1 * (-0.1333) * 1 = 0.0867$$

$$w_{ao} = w_{ao} + \alpha * \delta_o * o_a = 0.2 + 0.1 * (-0.1333) * 0.5050 = 0.1933$$

$$w_{bo} = w_{bo} + \alpha * \delta_o * o_b = -0.1 + 0.1 * (-0.1333) * 0.5695 = -0.1076$$

Tất cả các trọng số này sẽ dùng cho các mẫu kế tiếp.

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

➤ Thử tính cho “cây Iris A” thêm một bước kế tiếp, có kết quả:

$$O_a = f (-0.0007 + 0.0969*4.7 + (-0.1021)*3.2 + (-0.1009)*1.3 + (-0.0001)*0.2) = f (-0.0032) = 0.4992$$

$$O_b = f (0.0003 + -(0.0985)*4.7 + 0.2010*3.2 + 0.1004*1.3 + (-0.0999)*0.2) = f (0.2911) = 0.5724$$

$$O_o = f (0.0867 + 0.1933*0.4992 + (-0.1076)*0.5724) = f (0.1440) = 0.5304$$

➤ Kết quả, ta thấy 0.5304 gần về 0 (kết quả mong muốn) so với kết quả của bước trước đó là 0.5359.

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

Tóm tắt giải thuật

Cho trước K mẫu dữ liệu :

$$\left\{ \left(x^1, y^1 \right), \dots, \left(x^k, y^k \right) \right\}$$

với $x^k = (x_1^k, x_2^k, \dots, x_n^k)$ và $y^k \in \mathbb{R}$, $k = 1, \dots, K$.

Bước 1: chọn trước giá trị $\alpha > 0$ và $E_{\max} > 0$.

Bước 2: khởi tạo ngẫu nhiên w , bắt đầu với mẫu thứ nhất $k = 1$ và gán sai lệch $E = 0$.

Bước 3: bắt đầu quá trình học, gán $x = x^k$, $y = y^k$. Đầu ra của mạng neuron tính theo:

$$O^k = \frac{1}{1 + \exp(-W^T o^k)}$$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

trong đó o^k là giá trị ra của lớp ẩn, được tính bởi:

$$o_l^k = \frac{1}{1 + \exp(-w_l^T x_k)}$$

Bước 4: cập nhật trọng số neuron đầu ra: $W = W + \alpha \delta_o$

với $\delta = (y - O)O(1 - O)$

Bước 5: cập nhật trọng số của neuron ẩn

$$w_l = w_l + \alpha \delta W_l o_l (1 - o_l) x \quad (\text{cho } l = 1, \dots, L)$$

Tiếp cận mạng nơ-ron

- **Mạng lan truyền ngược (back propagation):**

Bước 6: tính sai lệch bằng cách cộng thêm sai lệch hiện tại

$$E = E + \frac{1}{2}(y - O)^2$$

Bước 7: nếu $k < K$ thì $k = k + 1$ và trở lại **Bước 3**. Nếu không thì qua **Bước 8**.

Bước 8: Nếu $E \leq E_{\max}$ thì kết thúc quá trình học. Còn nếu $E > E_{\max}$ gán $E = 0$ và bắt đầu một chu kỳ học mới bằng cách trở lại **Bước 3**.